



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Bhagyesh Patil** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher Name: Kajal

Joseph Signature:



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO 01

Name: Bhagyesh Patil

Class: D20A

Roll: No: 3

Batch: A

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Cryptographic Hash Function in Blockchain

A cryptographic hash function is a special mathematical technique that converts any kind of input data—such as transaction details, files, or text—into a fixed-size string of characters known as a hash. In blockchain technology, this hash works like a unique digital identity for the data stored inside a block. Even the smallest modification in the input, like changing one letter, produces a completely new hash value. This makes hashing extremely useful for ensuring blockchain security and trust. Blockchains such as Bitcoin and Ethereum commonly rely on hashing algorithms like SHA-256 to secure transaction records.

Simply put, hashing guarantees that once information is stored on a blockchain, it cannot be secretly changed without everyone noticing.

Features of Cryptographic Hash Functions

- **Consistency (Deterministic):** The same input always results in the exact same hash.
- **Constant Length Output:** Regardless of how big or small the input is, the hash output remains the same fixed size (for example, 256 bits in SHA-256).
- **Quick Processing:** Hash calculations are efficient and can be done rapidly, which is necessary for blockchain systems handling many transactions.
- **One-Way Nature (Pre-image Resistance):** It is nearly impossible to retrieve the original input from its hash.
- **Strong Uniqueness (Collision Resistance):** Two different inputs should not generate an identical hash.
- **Avalanche Property:** A minor input change leads to a drastically different hash result.

Importance of Hashing in Blockchain

- It links blocks by embedding the previous block's hash into the next one, creating a secure chain structure.
- It ensures immutability, since altering stored data would instantly change the hash and expose manipulation.
- It plays a major role in mining and transaction validation, helping networks confirm blocks before adding them.
- It strengthens protection of wallet addresses, digital signatures, and sensitive user information against fraud and tampering.

Properties of SHA-256

SHA-256 (Secure Hash Algorithm – 256-bit) belongs to the SHA-2 family and is one of the most commonly used hashing algorithms in blockchain technology. It converts any input data into a secure 256-bit hash value, regardless of the input's original size.

- **Fixed 256-bit Output:** SHA-256 always produces a hash of exactly 256 bits, usually shown as 64 hexadecimal characters, no matter how large the input is.
- **High Security Strength:** It is widely trusted for modern cryptography and is a standard choice in blockchain networks for strong protection.
- **Collision Resistance:** The chance that two different pieces of data generate the same hash is extremely rare, ensuring reliable data integrity.
- **Avalanche Property:** Even the smallest change in input data results in a completely different hash output, making unauthorized modifications easy to detect.
- **Irreversible (One-Way Function):** It is practically impossible to reverse the hash and recover the original input data.

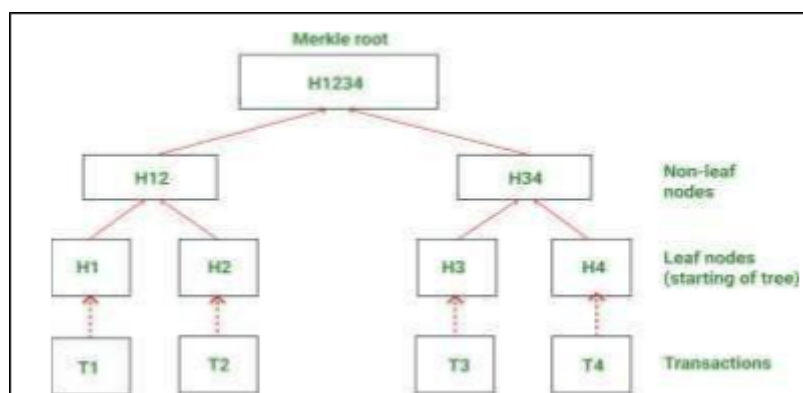
Merkle Tree (Hash Tree)

A Merkle Tree is a specialized binary tree structure used in blockchain to store and verify a large number of transactions efficiently. In this structure, every transaction is first converted into a hash, forming the leaf nodes. These hashes are then combined in pairs, hashed again, and repeated layer by layer until one final hash remains, called the **Merkle Root**.

The Merkle Root represents the complete set of transactions in a block. By storing only this single root hash inside the block header, blockchain systems can confirm that all transactions remain unchanged. If even one transaction is altered, the Merkle Root changes instantly, making fraud or tampering easy to detect.

Structure of a Merkle Tree

A Merkle Tree follows a layered hierarchical arrangement. It begins with individual transaction hashes at the bottom level. These hashes are merged pairwise to form parent nodes, continuing upward through multiple levels until reaching the topmost hash, the **Merkle Root**, which summarizes and secures all transactions within the block.



1. **Leaf Nodes:** These are the hashes of individual transactions in a block. Each transaction is first hashed to form a leaf node.
2. **Intermediate (Parent) Nodes:** Pairs of leaf nodes are combined and hashed again to form parent nodes. This process continues upward, combining child nodes at each level.
3. **Root Node (Merkle Root):** The topmost node of the tree, representing all transactions in the block. If any transaction changes, the Merkle Root changes, making tampering detectable.

2. Merkle Root

The Merkle Root is the **topmost hash** of the Merkle Tree. It acts as a summary of all transactions in a block.

If any transaction is changed, its hash changes, which affects the Merkle Root. Since the Merkle Root is stored in the block header, this makes tampering easy to detect.

3. Working of Merkle Tree

The Merkle Tree works by organizing and summarizing all transactions in a block into a single hash (the Merkle Root) so that data can be verified efficiently and securely.

The process follows these steps:

- **Hashing Transactions:** Each transaction in the block is first hashed using a cryptographic hash function, usually SHA-256. These hashes form the leaf nodes of the Merkle Tree.

Example:

Transactions: T1, T2, T3, T4 Hashes:

$H(T1)$, $H(T2)$, $H(T3)$, $H(T4)$

- **Pairing and Hashing:** The leaf nodes are grouped in pairs. Each pair of hashes is concatenated (joined together) and then hashed again to create a parent node. If the number of transactions is odd, the last hash is duplicated to form a pair. This ensures that every level of the tree has an even number of nodes.

Example:

- Pair 1: $H(T1) + H(T2) \rightarrow \text{Hash} = H12$
- Pair 2: $H(T3) + H(T4) \rightarrow \text{Hash} = H34$

- **Building the Tree:** The pairing and hashing process continues up the tree, combining parent nodes to create new higher-level nodes.

Example:

$H12 + H34 \rightarrow \text{Hash} = H1234$

- **Creating the Merkle Root:** This process repeats until only one hash remains at the top of the tree. This top-level hash is called the Merkle Root, which represents all transactions in the block.
- **Verification:** To verify that a transaction exists in a block, a Merkle Proof is used. Instead of checking every transaction, only a small number of hashes along the path from the transaction leaf to the Merkle Root are needed. This makes verification fast and efficient, even for large blocks of data. **Example:** To verify T1: Use H(T2) and H34 along with H(T1) to recalculate H1234. If it matches the Merkle Root, T1 is valid.
- **Tamper Detection:** If any transaction is modified, its hash changes. This change propagates up the tree and alters the Merkle Root. Since the Merkle Root is stored in the block header, any tampering becomes immediately obvious.
Example:
If T3 changes → H(T3) changes → H34 changes → H1234 changes → Merkle Root mismatch.

4. Benefits of Merkle Tree

- **Efficient Verification:** Only a small number of hashes are needed to verify a transaction using a Merkle Proof, so checking data is fast even for large blocks.
- **Data Integrity:** Any change in a transaction alters the Merkle Root, making it easy to detect tampering.
- **Reduced Storage:** Instead of storing all transaction data in the block header, only the Merkle Root is stored, saving space.
- **Scalability:** Merkle Trees allow blockchains to handle a large number of transactions without slowing down verification.
- **Security:** The structure ensures that transactions cannot be altered without being noticed, keeping the blockchain secure.

5. Use of Merkle Tree in Blockchain

- **Efficient Transaction Verification:** Nodes can verify a single transaction without downloading the entire block, using a Merkle Proof.
- **Ensures Data Integrity:** Any change in a transaction immediately changes the Merkle Root, helping detect tampering.
- **Reduces Storage Needs:** Only the Merkle Root is stored in the block header, instead of all transaction data, saving space.
- **Supports Lightweight Nodes:** Simple Payment Verification (SPV) nodes can confirm transactions securely without holding the full blockchain.

Applications of Merkle Tree

Merkle Trees are an important data structure used not only in blockchain but also in many digital security systems. Their main advantage is that they can verify huge amounts of data quickly without needing to store or process everything. Below are some common practical uses:

1. Blockchain Block Transaction Management

In cryptocurrencies such as Bitcoin, thousands of transactions may exist inside a single block. Merkle Trees help compress this information efficiently.

- **Example:** Instead of keeping the entire transaction list in the block header, only one hash value—the **Merkle Root**—is stored.
- If any transaction is modified, the Merkle Root changes instantly.

Why useful: Ensures block integrity and makes transaction verification simpler.

2. Lightweight Wallet Verification (SPV Method)

Many mobile wallets and small devices cannot store complete blockchain data. They use Merkle Trees for fast checking.

- **Example:** A phone wallet can verify a payment by downloading only the transaction proof path, not the entire block.

Benefit: Saves internet bandwidth, storage space, and still maintains trust.

3. Software Development and File Tracking

Merkle Trees are widely used in systems that need reliable tracking of file changes.

- **Example:** In software platforms, every file version can be hashed, and any update changes the root hash, clearly showing that the content was altered.

Advantage: Helps in managing project history and preventing unnoticed modifications.

4. Cloud Storage Validation

Cloud providers must guarantee that stored files are not corrupted over time.

- **Example:** A Merkle Tree can be built from file chunks, allowing users to verify that their uploaded data remains unchanged.

Result: Secure storage and easy corruption detection.

5. Secure Data Sharing in P2P Systems

Peer-to-peer networks often share data in fragments, making verification necessary.

- **Example:** When downloading a large video split into parts, Merkle Trees allow users to confirm each part is authentic before assembling the full file.

Benefit: Prevents fake or tampered file pieces from spreading.

6. Digital Auditing and Record Security

Merkle Trees are also useful in financial audits and secure record-keeping.

- **Example:** Banks or companies can store transaction logs in a Merkle Tree so that auditors can later confirm no records were changed.

Importance: Ensures transparency and long-term trust in stored logs.

Colab Notebook:

Code & Output:

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib

data_string = 'Bhagyesh-Patil' encoded_data = data_string.encode('utf-8') sha256_hash = hashlib.sha256(encoded_data) hex_digest = sha256_hash.hexdigest()

print(f"Original string: '{data_string}'")
print(f"SHA-256 Hash: {hex_digest}")
```

```
... Original string: 'Bhagyesh_Patil'
SHA-256 Hash: fedb6f6d2b3bfe8f8b6bd2390f95d2ad8fc926d830c352b5fdf232550f7876ed
```

2. Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```
import hashlib

input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

hash_string = input_string + nonce

hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

print("Hash Code:", hash_code)
```

```
Enter a string: Bhagyesh_Patil
Enter the nonce: 1
Hash Code: 2fe6f835deb82737db23d5fdd4b115dd015c8616540e76271ac375352c71f5fd
```

3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce hash
        string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash has required leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash: ", hash_code)
            return nonce

        nonce += 1

# Get input
input_string = "Bhagyesh Patil"
num_zeros = 1

# Find nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print result
print("Input String: ", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

```

Hash: 0 b271s a fa be c fa 5c a5 d 1 b91f4 1 a 2 B29B 1 dc b*0 6B27 d2da 6 2b2s 5357 c b25Bd be
Input String: Bhagyesh_Patil
Leading Zeros : 1
Expected Nonce : 20

```

4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```
import hashlib
def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()
def merkle_root(transactions):
    hashes = [sha256(tx) for tx in transactions]
    while len(hashes) > 1:
        if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])
        new_level = []
        for i in range(0, len(hashes), 2):
            combined_hash = hashes[i] + hashes[i + 1]
            new_level.append(sha256(combined_hash))
        hashes = new_level
    return hashes[0]
transactions = [
    "User1 sends 12 coins to User7", "Miner42
    rewards itself 6.25 coins", "WalletA
    transfers 0.8 coins to WalletB", "ContractX
    releases 15 coins to Vault9"
]
print("Merkle Root Hash:", merkle_root(transactions))
```

```
... Merkle Root Hash: 6f3d19cf44fda0e0a0af16ec5c6d00db12d88eeabc4b875b105db2d87b6b4eb1
```

Conclusion :

Cryptographic hash functions like **SHA-256** help keep blockchain data safe, secure, and unchangeable. **Merkle Trees** organize transactions in a way that makes it easy to verify them quickly and detect any tampering. Together, they make blockchain trustworthy, capable of handling large amounts of data without compromising security. These concepts help us understand how blockchain maintains security, efficiency even with large amounts of data.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.2

Name: Bhagyesh Patil

Class:D20A

Roll No:3

Batch:A

Aim: Create a Blockchain using Python

1. What is Blockchain

Blockchain is a revolutionary technology that functions as a shared, immutable digital ledger, often described as a decentralized database or distributed ledger technology. It records transactions across a network of computers in a way that ensures security, transparency, and resistance to tampering. The term "blockchain" derives from its structure: data is organized into blocks, each linked to the previous one through cryptographic hashes, forming a continuous chain. This decentralized nature eliminates the need for intermediaries, allowing multiple parties to perform transactions without third-party intervention.

Key characteristics include:

- **Decentralization:** No single entity controls the network; it's maintained by nodes (computers) that validate and record transactions.
- **Immutability:** Once data is added, it cannot be altered without consensus from the network, making it tamper-proof.
- **Transparency:** All participants can view the ledger, but privacy is maintained through cryptography.
- **Security:** Uses cryptographic techniques to protect data.

Blockchain stores various types of information, but it's most commonly used for transactions in cryptocurrencies like Bitcoin. It operates on principles such as consensus mechanisms to agree on the state of the ledger. Unlike traditional databases that structure data in tables, blockchain uses blocks

chained together, ensuring that changes to one block would require altering all subsequent blocks, which is computationally infeasible

2. What is a Block?

A block is the fundamental building block of a blockchain, acting as a container for data such as transactions. It serves as a record in the digital ledger, similar to a page in a book, where each block holds a complete record of transactions or other information. Blocks are linked together in a chain, with each new block referencing the previous one via a cryptographic hash, creating an unbreakable sequence.

In essence, a block is a node in the blockchain network that stores encrypted data, including a list of transactions, a timestamp, and a unique identifier (hash). The first block in any blockchain is called the Genesis Block, which has no previous block reference and is hardcoded into the protocol. This structure ensures the integrity of the entire chain, as altering any block would invalidate the hashes of all following blocks

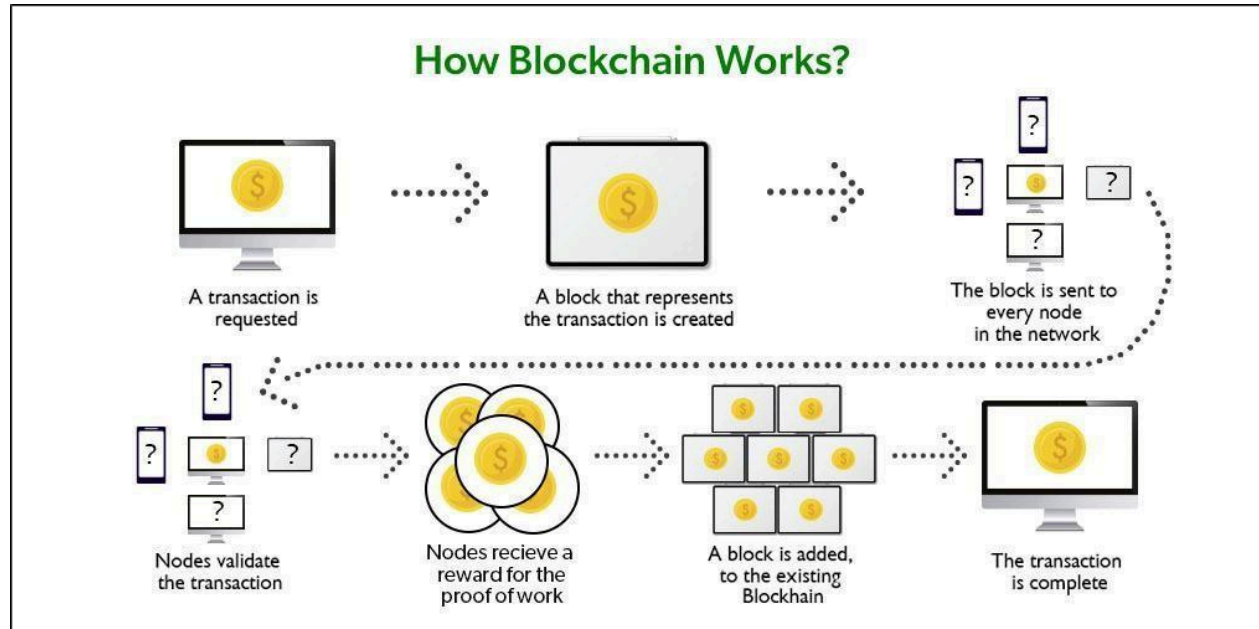
3. Components of a Block

A block in blockchain consists of two main parts: the block header and the block body (containing transactions). The header is crucial for identification and linking, while the body holds the actual data.

Key components include:

Component	Description
Block Header	Identifies the block and includes metadata like version, timestamp, Merkle root, difficulty target, nonce, and previous block hash.
Version	A 4-byte field tracking software or protocol upgrades.
Previous Block Hash	A 32-byte reference to the hash of the prior block, ensuring chain continuity.
Merkle Root	A 32-byte hash of all transactions in the block, derived from a Merkle tree for efficient verification.
Timestamp	A 4-byte record of the block's approximate creation time.
Difficulty Target	A 4-byte value setting the mining difficulty.
Nonce	A 4-byte number used in proof-of-work mining to find a valid hash.
Transactions	The list of verified transactions stored in the block body.

These elements work together to maintain security and enable validation. The header alone is often sufficient for light clients to verify blocks without the full data



4. Process of mining

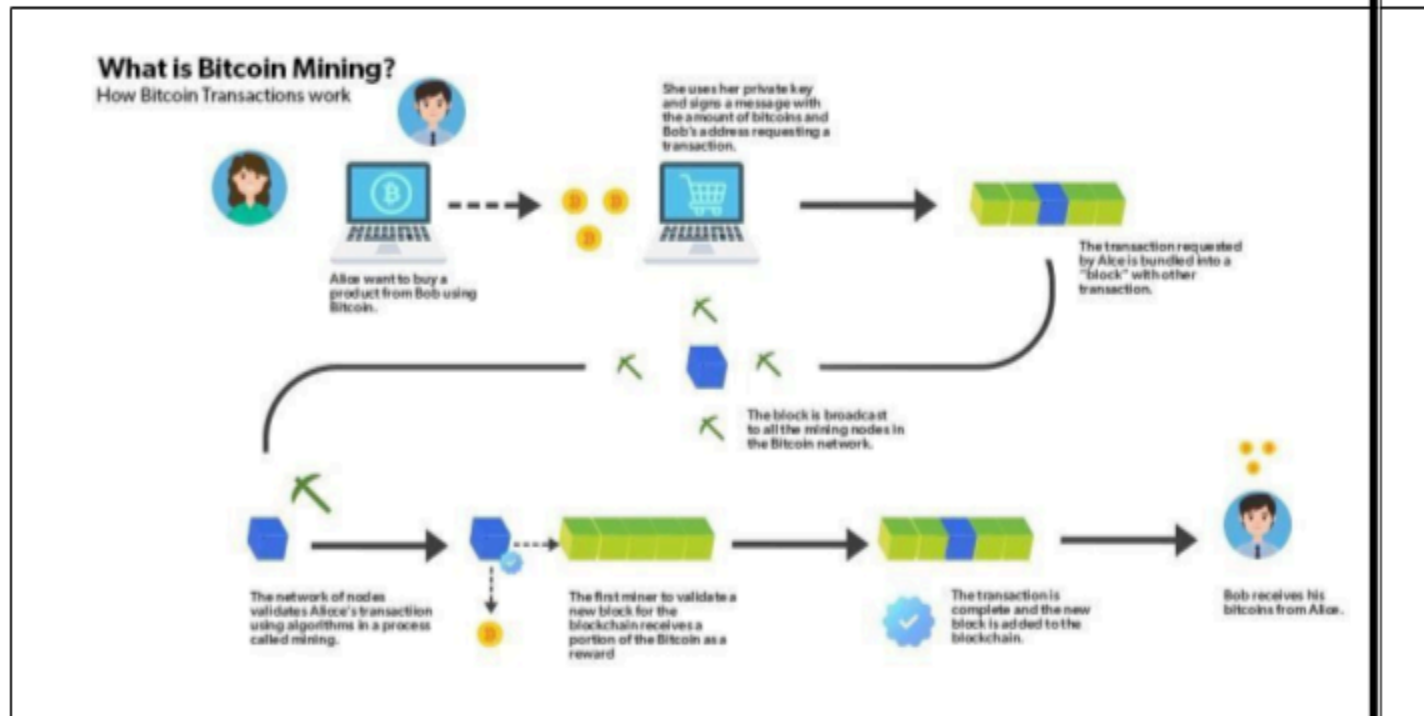
Mining is the process of validating transactions and adding new blocks to the blockchain. It is primarily associated with **Proof-of-Work (PoW)** consensus mechanisms (e.g., Bitcoin). Miners compete to solve a complex cryptographic puzzle, and the winner gets to add the block and receive a reward.

Key steps in the mining process:

1. **Transaction Collection** Unconfirmed transactions are collected from the network and stored in a **mempool** (memory pool). Miners select transactions (usually prioritizing those with higher fees) to include in a candidate block. A special **coinbase transaction** is added, which rewards the miner with new coins + transaction fees.

2. **Block Header Preparation** The miner constructs the block header, which includes:
 - Previous block hash
 - Merkle root (summary hash of all transactions)
 - Timestamp
 - Difficulty target
 - Nonce (a variable number starting from 0)
3. **Solving the Cryptographic Puzzle (Proof-of-Work)** Miners repeatedly hash the block header (using SHA-256 in Bitcoin) while changing the nonce value. The goal is to find a hash that is **less than or equal to the current difficulty target** (a very small number, often requiring the hash to start with many leading zeros). This is computationally intensive and probabilistic — miners try billions/trillions of nonces per second.
4. **Block Found** The first miner to find a valid nonce (producing a qualifying hash) has "solved" the puzzle. They broadcast the completed block to the network.
5. **Network Acceptance & Reward** Other nodes verify the block. If valid, it is added to their copy of the blockchain. The successful miner receives the **block reward** (e.g., currently 3.125 BTC as of recent halvings) + transaction fees. Difficulty adjusts periodically to keep average block time stable (~10 minutes in Bitcoin).

Mining secures the network by making it extremely expensive to alter past blocks (would require re-mining all subsequent blocks) and prevents double-spending.



5. Checking the validity of blocks in blockchain

Once a mined block is broadcast, every full node independently verifies it before accepting it into their blockchain copy. This decentralized validation is what makes blockchain trustless and secure.

Key validity checks:

1. Block Structure & Syntax

- Correct format and size
- Valid version number
- Timestamp is reasonable (not too far in future or before previous block)

2. Previous Block Hash Match

The "previous block hash" field must exactly match the hash of the most recent block already in the node's chain.

3. Proof-of-Work Validation Recompute the block hash → check if it is

$\leq$ current difficulty target (i.e., has enough leading zeros). This confirms the miner did the required work.

4. Merkle Root Verification Rebuild the Merkle tree from all transactions in the block body → verify the computed Merkle root matches the one in the block header.

5. Transaction Validity

- Each transaction has valid signatures
- Inputs are unspent (no double-spending)
- Input amounts $\geq$ output amounts + fees
- Transactions follow protocol rules

6. Consensus Rules Compliance Block follows all network rules (max block size, valid coinbase, etc.).

If **any** check fails, the block is rejected and not added to the chain. Nodes continue waiting for a valid block. The longest valid chain (most cumulative work) is considered the true chain.

Light clients (SPV wallets) use simplified checks — they rely on **Merkle proofs** to confirm a transaction exists in a block without downloading everything.

Implementation:

-Installing Flask

```
PS C:\Users\Student\Downloads\BCEx2> pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
```

blockchain.py

```
import datetime
```

```
import hashlib
```

```
import json
```

```
class Blockchain:
```

```
    def __init__
```

```
    (self):
```

```
        self.chain = []
```

```
        self.difficulty = 4 # 4 leading zeroes
```

```
        self.create_genesis_block()
```

```
    def create_genesis_block(self):
```

```
        genesis_block = {
```

```
            'index': 1,
```

```
            'timestamp': str(datetime.datetime.now()),
```

```
            'data': 'Genesis Block',
```

```
            'nonce': 0,
```

```
            'previous_hash': '0'
```

```

    }
    genesis_block['hash'] = self.calculate_hash(genesis_block)
    self.chain.append(genesis_block)

def calculate_hash(self, block):
    block_copy = block.copy()
    block_copy.pop('hash', None) # Remove hash before recalculating

    encoded = json.dumps(block_copy, sort_keys=True).encode()
    return hashlib.sha256(encoded).hexdigest()

def mine_block(self, data):
    previous_block = self.chain[-1]
    nonce = 0

    while True:
        block = {
            'index': len(self.chain) + 1,
            'timestamp':
                str(datetime.datetime.now()), 'data': data,
            'nonce': nonce,
            'previous_hash': previous_block['hash']
        }
        block_hash =
            self.calculate_hash(block) # Check
            difficulty condition
        if block_hash.startswith('0' *
            self.difficulty): block['hash'] =
            block_hash self.chain.append(block)
        return block

```

```
nonce += 1
```

```
def is_chain_valid(self):  
    for i in range(1, len(self.chain)):  
        current = self.chain[i]  
        previous = self.chain[i - 1]  
  
        # Check previous hash link  
        if current['previous_hash'] != previous['hash']:  
            return False  
  
        # Recalculate and verify hash  
        recalculated_hash =  
        self.calculate_hash(current) if  
        recalculated_hash != current['hash']:  
            return False  
  
        # Check difficulty rule  
        if not current['hash'].startswith('0' * self.difficulty):  
            return False  
  
    return True
```

app.py

```
from flask import Flask, jsonify  
from blockchain import Blockchain  
  
app = Flask(__name__)  
blockchain =  
Blockchain()  
  
@app.route('/mine_block', methods=['GET'])  
def mine_block():
```

```
block = blockchain.mine_block("Some transaction data")
return jsonify({
    'message': '⚡ Block mined successfully!',
    'block': block
}), 200
```

```
@app.route('/get_chain',
methods=['GET']) def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200
```

```
@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'is_valid': blockchain.is_chain_valid()
    }), 200
```

```
if __name__ == '__main__':
    print("⚡ Mining blockchain with 4 leading zeroes...")
    app.run(debug=True, use_reloader=False)
```



```

- 127.0.0.1:5000, 'mi
'pretty-print l°:e_block

{
  "index": 2,
  "message": "Congratulations, you just mined a block! ",
  "previous_hash":
  "2BSc6lBbfc5c342166a5e858c51591b12bBf7c82c02869f5d6ed34acB6e2bcb3",
  "proof" : 533,
  "timestamp": "2026-81-30 10: 13:56.814628"
}

```

```

← @ 127.0.0.1:5000, "ge' chain

{
  "previous_hash": "0",
  "proof": 1,
  "timestamp": "2826-81-38 18: 12: 35 . 588281"

  "index": 2,
  "previous_hash": "285c6tebfc 5c 342166a' e 858c 51591b12bsf7c82c0 2469-L-5d6ed 34acb6e 2bcb3 ", "proof":
  533,
  "timestamp": "2@26-61-36 18: 13: 56 . 81 46 28"

  "index": 3,
  "previous_hash":
  "852ec6442709d15e7d31d276283412823cd3697ae09ab866cae24e762903f38b", "proof":
  45293,
  "timestamp": "2826-01-30 10:23:20.985069"

  "index": 4,
  "previous_hash": " 591bde66577 8409 92f374 ce J 3t45669c f3195dea2 c6c 630c51 c f43858a4b9ab ", "proof":
  213 91,
  "timestamp": "2626-61-30 16: 23: 28 . 91 068 2"

  "index": 5,
  "previous_hash": " 24588966 eab937a 36b 6ba38db 36c428de145a234ff4dc d027bce2ab59925c296", "timestamp":
  "2626-61-50 16: 23: 30 . 49196 3"

  "index": 6,
  "previous_hash": "aaeb52083 c 0214ab 88 2t9 2461970f21334a6852c 8f 126500e 263dd2e66 a 22 b6",
  "proof": 46191,
  "timestamp": "2026-81-36 16: 25: 31. 2712 2 5"

  "length": 6
}

```



Conclusion:-

In this experiment, a simple blockchain was successfully built using Python. It clearly illustrated the essential concepts of blockchain, such as block structure, cryptographic hashing, Proof-of-Work mining, and chain validation. Through hash-linked blocks and strict difficulty-based verification rules, the system guarantees data integrity, immutability, and strong resistance to tampering. Even a minor change in any block breaks the entire chain, perfectly showcasing the decentralized and secure foundation of blockchain technology.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT 3

Name-Bhagyesh Patil

Class-D20A

Roll no-03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

A blockchain is a distributed, decentralized digital ledger used to record transactions securely across multiple computers (nodes). Instead of relying on a central authority, every node in the network maintains its own copy of the blockchain.

Each block in the blockchain contains:

- A list of transactions
- A timestamp
- The hash of the previous block
- Its own cryptographic hash

The blocks are linked together using hashes, forming a chain. Any attempt to modify data in a block changes its hash, which breaks the chain and makes tampering easily detectable. This property ensures data integrity, transparency, and immutability.

2. Mining

Mining is the process by which new blocks are added to the blockchain. It involves:

- Collecting pending transactions
- Creating a block header
- Solving a computationally difficult problem known as Proof-of-Work (PoW)

In Proof-of-Work, miners repeatedly change a nonce value to generate a hash that satisfies the network's difficulty requirement (such as leading zeroes). Once a valid hash is found, the block is added to the blockchain and shared with other nodes. As an incentive, miners receive a cryptocurrency reward for their computational effort.

3. Multi-Node Blockchain Network

In this experiment, a multi-node blockchain network is simulated using three nodes running on different ports (5001, 5002, and 5003).

Each node:

- Operates independently
- Maintains its own copy of the blockchain
- Communicates with peer nodes to share newly mined blocks

This setup demonstrates how blockchain achieves decentralization and synchronization without a central server.

4. Consensus Mechanism

To ensure consistency across all nodes, the Longest Chain Rule is used as the consensus mechanism.

When different versions of the blockchain exist, the node accepts the longest valid chain as the correct one. This rule ensures that all nodes eventually agree on a single transaction history, even if temporary differences arise.

5. Transactions and Mining Reward

Transactions in the cryptocurrency system include:

- Sender address
- Receiver address
- Transaction amount

When a block is mined, all pending transactions are added to the block. Additionally, a special transaction is created to reward the miner with newly generated cryptocurrency. This reward mechanism motivates miners to maintain and secure the network.

6. Chain Replacement

The chain replacement process ensures network consistency. When the `/replace_chain` endpoint is triggered:

- A node requests blockchain data from its peers
- Validates the received chains
- Replaces its own chain if a longer and valid chain is found.

This mechanism helps resolve conflicts and keeps all nodes synchronized.

Code:

```
# Module 2 - Create a Cryptocurrency
```

```
# To be installed:
```

```
# Flask==0.12.2: pip install Flask==0.12.2
```

```
# Postman HTTP Client: https://www.getpostman.com/
```

```

# requests==2.18.4: pip install requests==2.18.4

# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4
hex from urllib.parse import urlparse

# Generate a unique id that is in
# To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
        # Adding transactions before they
        # are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()
        # Set is used as there is no order to
        # be maintained as the nodes can be from all around the globe

    def create_block(self, proof,
                    previous_hash):
        block = {'index':
            len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions}
        # Adding transactions to make
        # the blockchain a cryptocurrency
        self.transactions = []
        # The list of transaction should
        # become empty after they are added to a block
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1

```

```

    check_proof = False
    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2
- previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] !=
            self.hash(previous_block): return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(str(proof**2
- previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] !=
            '0000': return False
        previous_block = block
        block_index += 1
    return True

# This method will add the transaction to the list of transactions
def add_transaction(self, sender, receiver, amount):
    self.transactions.append({'sender': sender,
                              'receiver': receiver,
                              'amount':
                              amount})
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1
# It will return the block index
to which the transaction should be added

```

```

# This function will add the node containing an address to the set of nodes created in init
function
def add_node(self, address):
    parsed_url = urlparse(address)          # urlparse will parse the url
from the address
    self.nodes.add(parsed_url.netloc)        # Add is used and not append
as it's a set. Netloc will only return '127.0.0.1:5000'

# Consensus Protocol. This function will replace all the shorter chain with the longer chain in
all the nodes on the network
def replace_chain(self):
    network = self.nodes                    # network variable is the set of
nodes all around the globe
    longest_chain = None                    # It will hold the longest chain
when we scan the network
    max_length = len(self.chain)            # This will hold the length of
the chain held by the node that runs this function
    for node in network:
        response = requests.get(f'http://{node}/get_chain')    # Use get chain
method already created to get the length of the chain
        if response.status_code == 200:
            length = response.json()['length']                  # Extract the length of the
chain from get_chain function
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):    # We check if the length
is bigger and if the chain is valid then
                max_length = length                                # We update the max length
                longest_chain = chain                              # We update the longest chain
            if longest_chain:                                     # If longest_chain is not none that
means it was replaced
                self.chain = longest_chain                        # Replace the chain of the
current node with the longest chain
                return True
            return False                                         # Return false if current chain is
the longest one
# Part 2 - Mining our

```

Blockchain # Creating a Web

App

```
app = Flask(__name__)
```


Creating an address for the node on Port 5000. We will create some other nodes as well on different ports

```
node_address = str(uuid4()).replace('-', '') #
```

Creating a Blockchain

```
blockchain = Blockchain()
```

Mining a new block

```
@app.route('/mine_block', methods = ['GET'])
```

```
def mine_block():
```

```
    previous_block = blockchain.get_previous_block()
```

```
    previous_proof = previous_block['proof']
```

```
    proof = blockchain.proof_of_work(previous_proof)
```

```
    previous_hash = blockchain.hash(previous_block)
```

```
    blockchain.add_transaction(sender = node_address, receiver = 'Abcde', amount = 1) #
```

Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block

```
    block = blockchain.create_block(proof, previous_hash)
```

```
    response = {'message': 'Congratulations, you just mined a block!',
```

```
                'index': block['index'],
```

```
                'timestamp': block['timestamp'],
```

```
                'proof': block['proof'],
```

```
                'previous_hash': block['previous_hash'],
```

```
                'transactions': block['transactions']}
```

```
    return jsonify(response), 200
```

Getting the full Blockchain

```
@app.route('/get_chain', methods = ['GET'])
```

```
def get_chain():
```

```
    response = {'chain': blockchain.chain,
```

```
                'length': len(blockchain.chain)}
```

```
    return jsonify(response), 200
```

Checking if the Blockchain is valid

```
@app.route('/is_valid', methods =
```

```
['GET']) def is_valid():
```

```
    is_valid = blockchain.is_chain_valid(blockchain.chain)
```

```
    if is_valid:
```

```
        response = {'message': 'All good. The Blockchain is valid.'}
```

else:

```
    response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200
```

Adding a new transaction to the Blockchain

```
@app.route('/add_transaction', methods = ['POST'])
```

 # Post method as we have to pass something to get something in return

```
def add_transaction():
```

```
    json = request.get_json()
```

 # This will get the json file from postman. In Postman we will create a json file in which we will pass the values for the keys in the json file

```
    transaction_keys = ['sender', 'receiver', 'amount']
```

```
    if not all(key in json for key in transaction_keys):
```

 # Checking if all keys are available in json

```
        return 'Some elements of the transaction are missing', 400
```

```
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
```

```
    response = {'message': f'This transaction will be added to Block {index}'} 
```

```
    return jsonify(response), 201
```

 # Code 201 for

creation # Part 3 - Decentralizing our Blockchain

Connecting new nodes

```
@app.route('/connect_node', methods = ['POST'])
```

 # POST request to register the new nodes from the json file

```
def connect_node():
```

```
    json = request.get_json()
```

```
    nodes = json.get('nodes')
```

 # Get the nodes from json

```
    if nodes is None:
```

```
        return "No node", 400
```

```
    for node in nodes:
```

```
        blockchain.add_node(node)
```

```
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:',
```

```
                'total_nodes':
```

```
                list(blockchain.nodes)} return
```

```
    jsonify(response), 201
```

Replacing the chain by the longest chain if

```
needed @app.route('/replace_chain', methods = ['GET']) def replace_chain():
```

```
    is_chain_replaced = blockchain.replace_chain()
```

```

    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
                    'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 5003)

```

Output:

1)/get_chain

This GET request retrieves the complete blockchain from the node, showing all mined blocks and stored transactions.

The screenshot shows a REST client interface with the following details:

- Request:** GET http://127.0.0.1:5001/get_chain
- Response Status:** 200 OK
- Response Time:** 1.93 s
- Response Size:** 291 B
- Response Body (JSON):**

```

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-11 23:46:01.894114",
      "transactions": []
    }
  ],
  "length": 1
}

```

2)/mine_block

This GET request performs the mining process using Proof-of-Work and adds a new block to the blockchain.

GET

http://127.0.0.1:5001/mine_block

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Key	Value	Description
Key	Value	Description

BodyCookiesHeaders (5)Test Results

200 OK • 363 ms • 463 B •

{ } JSON

Preview

Visualize

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8616d710a286c411509a7d2a7",
5   "proof": 533,
6   "timestamp": "2026-02-12 00:13:54.652535",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Richard",
11      "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
12    }
13  ]
14 }
```

GET

http://127.0.0.1:5001/get_chain

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description
Key	Value	Description

BodyCookiesHeaders (5)Test Results

200 OK • 137 ms • 534 B •

{ } JSON

Preview

Visualize

```
4   "index": 1,
5   "previous_hash": "0",
6   "proof": 1,
7   "timestamp": "2026-02-11 23:46:01.894114",
8   "transactions": []
9 },
10 {
11   "index": 2,
12   "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8616d710a286c411509a7d2a7",
13   "proof": 533,
14   "timestamp": "2026-02-12 00:13:54.652535",
15   "transactions": [
16     {
17       "amount": 1,
18       "receiver": "Richard",
19       "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
20     }
21   ]
22 }
```

3)/is_valid

This GET request checks whether the current blockchain is valid and untampered.

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/is_valid`. The method is set to **GET**. The response status is **200 OK** with a response time of 74 ms and a body size of 215 B. The response body is displayed in JSON format:

```
{
  "message": "All good. The Blockchain is valid."
}
```

The interface includes tabs for Docs, Params, Authorization, Headers (7), Body, Scripts, and Settings. A table for headers is visible with columns for Key, Value, and Description.

4)/add_transaction

This POST request submits a new transaction (sender, receiver, amount) to be added to the next mined block.

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/add_transaction`. The method is set to **POST**. The request body is displayed in JSON format:

```
{
  "sender": "Bhagyesh",
  "receiver": "bhagyesh12",
  "amount": 500
}
```

The interface includes tabs for Params, Authorization, Headers (8), Body, Pre-request Script, Tests, and Settings. Below the tabs, there are radio buttons for content types: none, form-data, x-www-form-urlencoded, raw, binary, and JSON (selected). A Beautify button is also present.

5)/connect_node

This POST request connects the current node with other peer nodes in the blockchain network.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:5001/connect_node
- Body:** A JSON object with a "nodes" array containing two URLs: "http://127.0.0.1:5000" and "http://127.0.0.1:5002".
- Response:** 201 CREATED, 121 ms, 326 B. The response body is a JSON object with a "message" and a "total_nodes" array.

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5000",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5000",
5     "127.0.0.1:5002"
6   ]
7 }
```

6)/replace_chain

This GET request checks all connected peer nodes and replaces the current blockchain with the longest valid chain, ensuring network-wide consistency.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:5001/replace_chain
- Response:** 200 OK, 8.38 s, 582 B. The response body is a JSON object with an "actual_chain" array containing two block objects.

```
1 {
2   "actual_chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-11 23:46:01.894114",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8616d710a286c411509a7d2a7",
13      "proof": 533,
14    }
15  ]
16 }
```

```
{
  "transactions": [
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
    }
  ],
  "message": "All good. The chain is the largest one."
}
```

Conclusion:

In this experiment, a basic cryptocurrency system was designed and implemented using Python by applying fundamental blockchain principles such as block generation, cryptographic hashing, Proof-of-Work mining, transaction processing, and decentralized networking. A Flask-based framework was used to enable communication between multiple nodes, with each node maintaining its own copy of the blockchain ledger.

The mining process required solving the Proof-of-Work challenge, after which new blocks were added to the chain and miners received rewards. The system also supported transaction validation and inclusion of verified transactions into newly mined blocks. Through this implementation, the experiment offered hands-on insight into the functioning of blockchain technology, demonstrating how consensus, decentralization, and mining together enable the secure operation of cryptocurrencies in distributed environments.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT NO.4

Name:Bhgyesh Patil

Class:D20A

Roll No:3

Batch:A

Aim:Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:-

1. Primitive Data Types, Variables, Functions - pure,

view Primitive (Value) Data Types in Solidity include:

- bool → true/false
- int / uint (signed/unsigned integers, e.g., uint256 is most common)
- address → 20-byte Ethereum address
- bytes1 to bytes32 → fixed-size byte arrays
- string → dynamic UTF-8 encoded text (reference type, but often grouped here)

Variables are declared with a type and can be:

- **State variables** → stored permanently on the blockchain (expensive)
- **Local variables** → exist only during function execution

Functions can be marked as:

- pure → does not read or write state (computes only from inputs; cheapest gas)
 - view → reads state but does not modify it (e.g., getters; no gas when called externally)

Example

```
function getResult() public view returns (uint product, uint sum) {  
    product = num1 * num2; // reads state  
    sum = num1 + num2;  
}
```

```
function pureCalc(uint a, uint b) public pure returns (uint) {  
    return a + b; // no state access  
}
```

2. Inputs and Outputs to Functions

Functions in Solidity can take **parameters** (inputs) and return **values** (outputs).

- **Inputs:** Declared in parentheses; can use data locations like memory or calldata for reference types.
 - **Outputs:** Declared after returns keyword; can return multiple values.

Example:

```
function add(uint a, uint b) public pure returns (uint sum) {  
    sum = a + b;  
}  
  
// Multiple outputs  
  
function getValues() public view returns (uint, string memory) {  
    return (age, name);  
}
```

- Use calldata for external calls (cheaper, read-only).
- Use memory for temporary copies inside functions.

3. Visibility, Modifiers and Constructors

Visibility Specifiers (for functions and state variables):

- **public** → anyone can call/read (default for state vars creates getter)
- **private** → only inside current contract
- **internal** → current + derived (child) contracts
- **external** → only external calls (cheaper for large data)

Modifiers → reusable code blocks that run before/after function body (e.g., access control).

Use `_;` to insert function body.

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}
```

```
function restricted() public onlyOwner { ... }
```

Constructors → special function that runs once on deployment (initializes state).

- Syntax: `constructor() { ... }` (older: same name as contract)

4. Control Flow: if-else, loops

Solidity supports standard control structures:

if-else:

```
if (condition) {  
    // true  
} else if (another) {  
    // else-if  
} else {  
    // false  
}
```

Loops:

- for (uint i = 0; i < 10; i++) { ... }
- while (condition) { ... }
- do { ... } while (condition);

Avoid unbounded loops (gas limit risk). Use break / continue when needed.

5. Data Structures: Arrays, Mappings, structs, enums

- **Arrays**
 - Fixed: uint[5] arr;
 - Dynamic: uint[] arr; (use .push(), .pop(), .length)
- **Mappings** → key-value store (like hash table) mapping(address => uint) public balances;

Keys can be most types; no iteration possible.

Structs → custom composite types

```
struct User {  
    address wallet;  
    uint balance;  
    bool active;  
}
```

User public owner;

Enums → named constants (integers under the hood)

```
enum Status { Pending, Active, Cancelled }  
Status public state = Status.Pending;
```

6. Data Locations

Solidity has three main **data locations** for reference types (arrays, structs, mappings, strings):

- **storage** → permanent blockchain storage (persistent, expensive) Default for state variables.
 - **memory** → temporary, function lifetime only (deleted after execution)
Cheap; used for local variables & function args/returns.
- **calldata** → read-only, non-modifiable area for function call data
Cheapest for external function parameters (immutable copy of tx data).

Rule:

- State vars → always storage
- Function args → prefer calldata (external) or memory
- Local reference vars → must specify location

7. Transactions: Ether and wei, Gas and Gas Price, Sending Transactions

- **Ether units** → smallest is **wei** ($1 \text{ ETH} = 10^{18} \text{ wei}$) Other: **gwei** (10^9 wei), commonly used for gas prices.
 - **Gas** → computational effort unit
 - **Gas Limit** → max gas willing to spend (set by sender)
 - **Gas Price** → price per gas unit (in wei/gwei; set by sender)
 - Total fee = gas used $\times$ gas price
 - **msg.value** → amount of wei sent with transaction
- **Sending Ether** → use `.transfer()`, `.send()`, or `.call{value: amount}("")` (recommended low-level call)

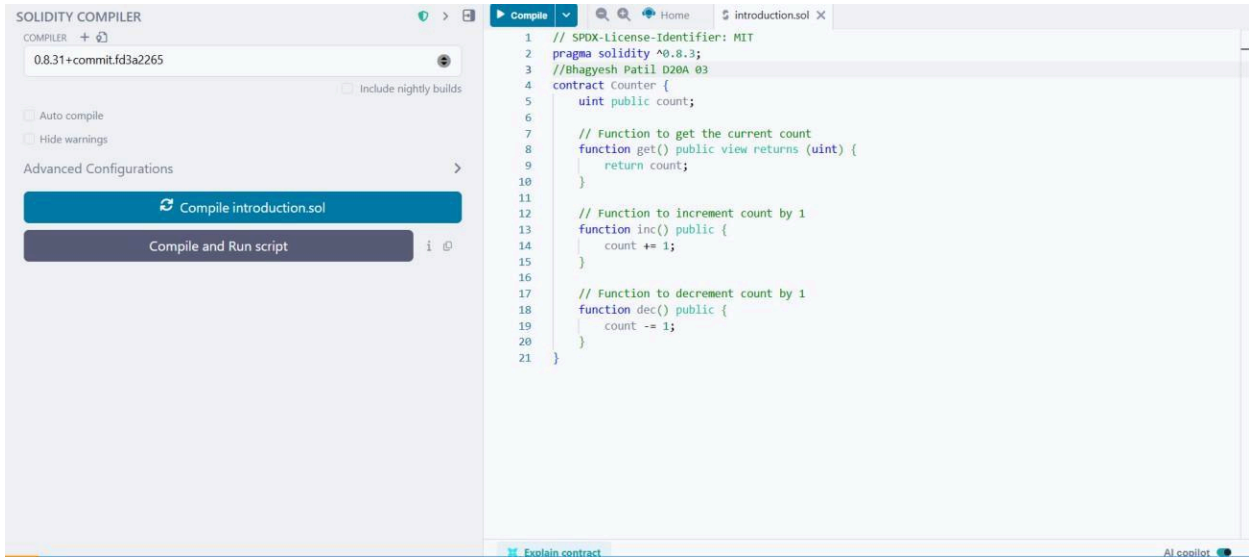
Example:

```
function sendEther(address payable recipient) public payable {  
    recipient.transfer(msg.value);  
}
```

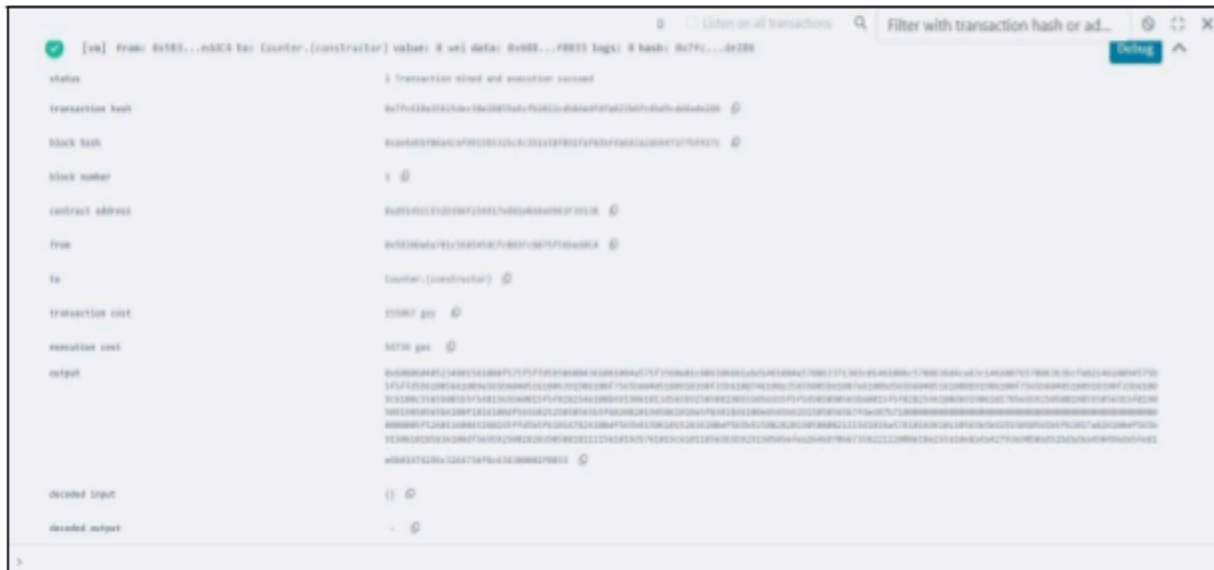
- Use payable for addresses/functions that receive Ether.
- `tx.gasprice` → current gas price (global var)

1

- Tutorial no. 1 – Compile the code



- Tutorial No.1-Deploy the contract



- Tutorial no. 1 – get

msg	[call] from: 0x5830da701c568545dcfc803fc8075f56beddc4 to: Counter.get() data: 0xd4...ce63c
from	0x5830da701c568545dcfc803fc8075f56beddc4
to	Counter.get() 0xd9145cc520386f254917e481e944e9943f39138
execution cost	2453 gas (Cost only applies when called by a contract)
input	0xd4...ce63c
output	0x00
decoded input	()
decoded output	{ "0": "uint256: 0" }
logs	()
raw logs	()

Deployed Contracts

COUNTER AT 0XD91...39138

Balance: 0 ETH

dec

inc

count

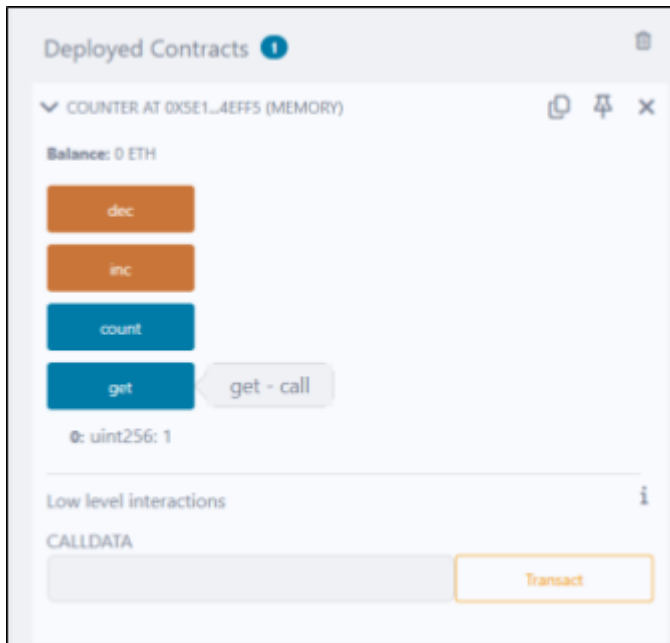
get

Low level interactions

CALLDATA

Transact

- Tutorial no. 1 – Decrement



- Tutorial no.2

LEARNETH

Tutorials list

Syllabus

2. Basic Syntax

2 / 19

2. Basic Syntax

In this section, we will create our first *smart contract*. This contract only consists of a string that holds the value "Hello World!".

In the first line, we should specify the license that we want to use. You can find a comprehensive list of licenses here: <https://spdx.org/licenses/>.

Using the `pragma` keyword (line 3), we specify the Solidity version we want the compiler to use. In this case, it should be greater than or equal to `0.8.3` but less than `0.9.0`.

We define a contract with the keyword `contract` and give it a name, in this case, `HelloWorld` (line 5).

Inside our contract, we define a *state variable* `greet` that holds the string `"Hello World!"` (line 6).

Solidity is a *statically typed* language, which means that you need to specify the type of the variable when you declare it. In this case, `greet` is a `string`.

We also define the *visibility* of the variable, which specifies from where you can access it. In this case, it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the *creator* of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

Compile

Home

introduction.sol

basicSyntax.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Bhagyesh Patil 03
4 contract HelloWorld {
5
6     string public name = "Alice";
7
8 }
```

- Tutorial no.3

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types

3 / 19

3. Primitive Data Types

In this section, we will show you Solidity's primitive data types, how to declare them, and their characteristics.

bool

You can declare data a boolean type by using the keyword 'bool'. Booleans can either have the value `true` or `false`.

uint

We use the keywords `uint` and `uint8` to `uint256` to declare an *unsigned integer type* (they don't have a sign, unlike `-12`, for example). Uints are integers that are positive or zero and range from 8 bits to 256 bits. The type `uint` is the same as `uint256`.

int

We use the keywords `int` and `int8` to `int256` to declare an integer type. Integers can be positive, negative, or zero and range from 8 bits to 256 bits. The type `int` is the same as `int256`.

address

Variables of the type `address` hold a 20-byte value, which is the size of an Ethereum address. There is also a special kind of Ethereum address, `address payable`, which can receive ether from the contract.

All these data types have default values, as shown in the contract (line 29).

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in

Compile

introduction.sol basicSyntax.sol primitiveDataTypes.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Bhagyesh Patil 03
4 contract Primitives {
5
6     bool public boo = true;
7
8     uint public u = 123;
9
10    int public i = -123;
11
12    address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733C;
13
14    // Assignment variables
15
16    address public newAddr = 0x0000000000000000000000000000000000000000000000000000000000000000;
17
18    int public neg = -5;
19
20    uint8 public newU = 0;
21
22 }
```

- Tutorial no.4

LEARNETH

Tutorials list

Syllabus

4. Variables

4 / 19

4. Variables

There are three different types of variables in Solidity: *State Variables*, *Local Variables*, and *Global Variables*.

1. State Variables

State Variables are stored in the contract storage and thereby on the blockchain. They are declared inside the contract but outside the function. This contract has two state variables, the string `text` (line 6) and the uint `num` (line 7).

2. Local Variables

Local Variables are stored in the *memory* and their values are only accessible within the function they are defined in. Local Variables are not stored on the blockchain. In this contract, the uint `i` (line 11) is a local variable.

3. Global Variables

Global Variables, also called *Special Variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the Solidity documentation.

Compile

introduction.sol basicSyntax.sol primitiveDataTypes.sol variables.sol 3

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Bhagyesh Patil 03
4 contract Variables {
5
6     string public text = "Hello";
7     uint public num = 123;
8
9     uint public blockNumber;
10    function doSomething() public { 22334 gas
11
12        uint i = 456;
13
14        uint timestamp = block.timestamp;
15        address sender = msg.sender;
16
17        blockNumber = block.number;
18    }
19
20 }
```

• Tutorial no.5

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

pure. For example, in this contract, the **get** function (line 14) marked **view** that only returns **num** does not change the state.

To define a function, use the **function** keyword followed by a unique name.

If the function takes inputs like our **set** function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them **view** or **pure** as we do for the **get** function if they don't modify the state. Our **get** function also returns values, so we have to specify the return types. In this case, it's a **uint**, since the state variable **num** that the function returns is a **uint**.

We will explore the particularities of Solidity functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

★ Assignment

1. Create a public state variable called **b** that is of type **bool** and initialize it to **true**.
2. Create a public function called **get_b** that returns the value of **b**.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

asicSyntax.sol

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

1 // Bhagyesh Patil 03

2 // SPDX-License-Identifier: MIT

3 pragma solidity ^0.8.3;

4

5 contract SimpleStorage {

6

7 uint public num;

8

9 bool public b = true;

10

11 function set(uint _num) public { 22536 gas

12 num = _num;

13 }

14

15 function get() public view returns (uint) { 2475 gas

16 return num;

17 }

18

19 function get_b() public view returns (bool) { 2539 gas

20 return b;

21 }

22 }

Explain contract

AI copilot

• Tutorial no.6

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

3. Accessing any of the members of block, tx, msg (with the exception of **msg.sig** and **msg.data**).
4. Calling any function not marked **pure**.
5. Using inline assembly that contains certain opcodes."

From the [Solidity documentation](#).

You can declare a pure function using the keyword **pure**. In this contract, **add** (line 13) is a pure function. This function takes the parameters **i** and **j**, and returns the sum of them. It neither reads nor modifies the state variable **x**.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions **view** and **pure** can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ Assignment

Create a function called **addToX** that takes the parameter **y** and updates the state variable **x** with the sum of the parameter and the state variable **x**.

Check Answer

Show answer

Next

Well done! No errors.

Compile

types.sol

variables.sol 3

readAndWrite.sol

viewAndPure.sol

1 // Bhagyesh Patil https://remix.ethereum.org/plugins/learneth/index.html/detail?id=1030

2 // SPDX-License-Identifier: MIT

3 pragma solidity ^0.8.3;

4

5 contract ViewAndPure {

6

7 uint public x = 1;

8

9 function addToX(uint y) public view returns (uint) { Infinite gas

10 return x + y;

11 }

12

13 function add(uint i, uint j) public pure returns (uint) { Infinite gas

14 return i + j;

15 }

16

17 function addToX2(uint y) public { Infinite gas

18 x = x + y;

19 }

20 }

Explain contract

AI copilot

RemixAI Copilot (enabled)

Initialize as git repo

Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.

• Tutorial no.7

Tutorial > Data Types - Charity

Data Types - Charity

This is how farst the data types - or charity, tunool state variable all the data types. There on results st tes known as therdmard having the something.

You can need to acmot and antrate data types from eritish cating the dynamic of the different LearnETH.

Assignment

- Data Types - Charity
- Data Types - structs
- Data Types - types
- Data Types - visibility
- Data Types - data language

Check Answer

Show answer

Next

Well done! No errors.

```

48     string security location;
49     string solidity struct;
50     string external histoner;
51     string public visibility;
52     string external myExternalVar = "Bhagyesh Patil D20A 03";
53     string external myExternalVar ;
54     string external myExternalVar ;
55     string public donorName = "Bhagyesh Patil D20A 03";
56     string public donorName;
57     string public donorName;
58     string external donorName;
59     string public myExternaVarVar;
60     string public visibility;
61 }

```

Salagmoetit Sockhun string /LearnETH\loannt> sd!
 Inoacting locations string: types...
 Exitution: Checked.

Data Types- myExternalVar
 donor Name =

C:\Terminal Sockhun string /LearnETH\loannt> █

• Tutorial no.8

LEARNETH

< Tutorials list Syllabus

< 5.4 Functions - Inputs and Outputs 8 / 19 >

parameters of contract functions that are publicly visible. from the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `1` and `true` without using a return statement.

Check Answer Show answer

Next

Well done! No errors.

```

71     function arrayInput(uint[] memory _arr) public {} Infinite gas
72
73     // Can use array for output
74     uint[] public arr;
75
76     function arrayOutput() public view returns (uint[] memory) { Infinite gas
77         return arr;
78     }
79
80     //Aryan Patankar D20A 38
81     function returnTwo() public pure returns( 472 gas
82         int i,
83         bool b
84     )
85     {
86         i=-2;
87         b=true;
88     }

```

Explain contract

0 Listen on all transactions Filter with transaction hash

Type the library name to see available commands.

- Tutorial no.9

LEARNETH

6. Visibility
9 / 19

and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `testPrivateFunc` directly. You will only be able to call them via `testInternalFunc` and `testInternalVar`.

Watch a video tutorial on Visibility

★ **Assignment**

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer **Show answer**

Next

Well done! No errors.

```

52 // string external externalVar = "my external variable";
53 }
54
55 contract Child is Base {
56 // Inherited contracts do not have access to private functions
57 // and state variables.
58 // function testPrivateFunc() public pure returns (string memory) {
59 //     return privateFunc();
60 // }
61
62 // Internal function call be called inside child contracts.
63 //Aryan Patankar D20A 38
64 function testInternalFunc() public pure override returns (string memory) { Infinite gas
65     return internalFunc();
66 }
67
68 function testInternalVar() public view returns(string memory x,string memory y){ Infinite gas
69     return(internalVar,publicVar);
70 }
71 }

```

Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

Type the library name to see available commands.

- Tutorial no.10

LEARNETH

7.1 Control Flow - If/Else
10 / 19

the condition of the `if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

★ **Assignment**

Create a new function called `evenCheck` in the `Base` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer **Show answer**

Next

Well done! No errors.

```

10 } else {
11     return 2;
12 }
13 }
14
15 function ternary(uint _x) public pure returns (uint) { Infinite gas
16 // if (_x < 10) {
17 //     return 1;
18 // }
19 // return 2;
20
21 // shorthand way to write if / else statement
22 return _x < 10 ? 1 : 2;
23 }
24 //Aryan Patankar D20A 38
25 function evenCheck(uint _number) public pure returns (bool) { Infinite gas
26     return (_number % 2 == 0) ? true : false;
27 }
28 }

```

Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

Type the library name to see available commands.

- Tutorial no.11

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

[Watch a video tutorial on Loop statements.](#)

★ Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

modifiersAndConstructors.sol

inputsAndOutputs.sol

```

15         }
16         }
17         count++;
18     }
19
20     // while loop
21     //Bhagyesh Patil D20A 03
22     uint j;
23     while (j < 10) {
24         j++;
25         if (count < 9) {
26             count++;
27         }
28     }
29 }
30
31

```

Explain contract

0

Listen on all transactions

Filter

Type the library name to see available commands.

- Tutorial no.12

LEARNETH

Tutorials list

Syllabus

8.1 Data Structures - Arrays

12 / 19

the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Constructors.sol

inputsAndOutputs.sol

visibility.sol

ifElse

```

4     contract Array {
5         // Several ways to initialize an array
6         //Bhagyesh Patil D20A 03
7         uint[] public arr;
8         uint[] public arr2 = [1, 2, 3];
9         uint[3] public arr3 = [0, 1, 2];
10
11         // Fixed sized array, all elements initialize to 0
12         uint[13] public myFixedSizeArr;
13
14         function get(uint i) public view returns (uint) {
15             return arr[i];
16         }
17
18
19         // Solidity can return the entire array.
20         // But this function should be avoided for
21         // arrays that can grow indefinitely in length.
22         function getArr() public view returns (uint[3] memory) {
23             return arr3;
24         }
25     }

```

Explain contract

0

Listen on all transactions

Filter with trans

Type the library name to see available commands.

• Tutorial no.13

LEARNETH

Tutorials list

Syllabus

8.2 Data Structures - Mappings

12 / 19

with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

Compile

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.2;
3 //Bhagyesh Patil 028A 03
4
5 contract Mapping {
6     // Mapping from address to uint
7     mapping(address => uint) public balances;
8
9     function get(address _addr) public view returns (uint) {
10         // Mapping always returns a value.
11         // If the value was never set, it will return the default value.
12         return balances[_addr];
13     }
14
15     function set(address _addr) public {
16         // Update the value at this address
17         balances[_addr] = _addr.balance;
18     }
19
20     function remove(address _addr) public {
21         // Reset the value to the default value.
22         delete balances[_addr];
23     }
24 }

```

15 Explain contract

0

Listen on all transactions

Filter with trans

Type the library name to see available commands.

• Tutorial no.14

LEARNETH

Tutorials list

Syllabus

8.3 Data Structures - Structs

13 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `Todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Compile

```

35
36
37 // update text
38 function update(uint _index, string memory _text) public {
39     Todo storage todo = todos[_index];
40     todo.text = _text;
41 }
42
43 // update completed
44 function toggleCompleted(uint _index) public {
45     Todo storage todo = todos[_index];
46     todo.completed = !todo.completed;
47 }
48 //Bhagyesh Patil 028A 00
49 function toggleCompleted(uint _index) public {
50     Todo storage todo = todos[_index];
51     todo.completed = !todo.completed;
52 }
53
54 // update remove
55 function remove(uint _index) public {
56     delete todos[_index];
57 }

```

15 Explain contract

0

Listen on all transactions

Filter with transaction has

Type the library name to see available commands.

- Tutorial no.15

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

14

// default value is the first element listed in

15

// definition of the type, in this case "Pending"

16

Status public status;

17

18

//Aryan Patankar D26A 38

19

enum Size {

20

S,

21

M,

22

L

23

}

24

25

Size public sizes;

26

27

function getSize() public view returns (Size) {

28

return sizes;

29

}

30

31

32

// Returns uint

33

// Pending - 0

34

// Shipped - 1

35

// Accepted - 2

36

// Pending - 0

Explain contract

0

Listen on all transactions

Filter with transaction hash or address

Type the library name to see available commands.

- Tutorial no.16

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `mystruct` member `foo` inside the `function f` to 4.
2. Create a new struct `myNewStruct2` with the data location memory inside the `function f` and assign it the value of `mystruct`. Change the value of the `myNewStruct2` member `foo` to 1.
3. Create a new struct `myNewStruct3` with the data location memory inside the `function f` and assign it the value of `mystruct`. Change the value of the `myNewStruct3` member `foo` to 3.
4. Let the function `f` return `mystruct`, `myNewStruct2`, and `myNewStruct3`.

Check Answer

Show answer

Tip: Make sure to create the correct return types for the function `f`.

12

//Aryan Patankar D26A 38

13

function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {

14

// call _f with state variables

15

_f(arr, map, mystructs[1]);

16

17

// 1. get a struct from a mapping (Storage reference)

18

MyStruct storage myStruct = mystructs[1];

19

// change the value of the mystruct member foo to 4

20

myStruct.foo = 4;

21

22

// 2. create a struct in memory

23

MyStruct memory myNewStruct = MyStruct(0);

24

25

// Create myNewStruct2 (Memory reference)

26

MyStruct memory myNewStruct2 = myNewStruct;

27

// change myNewStruct2 member foo to 1

28

myNewStruct2.foo = 1;

29

30

// 3. Create myNewStruct3 (Independent copy from storage)

31

MyStruct memory myNewStruct3 = myStruct;

32

myNewStruct3.foo = 3;

33

34

// 4. Return all three structs

35

return (myStruct, myNewStruct2, myNewStruct3);

36

}

Explain contract

0

Listen on all transactions

Filter with transaction hash or address

Type the library name to see available commands.

• Tutorial no.17

LEARNETH

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

to specify a unit or `ether`, we can add the suffixes `wei`, `gwei`, or `ether` to a literal number.

`wei`

Wei is the smallest subunit of *Ether*, named after the cryptographer Wei Dai. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

Home

sendingEther.sol 3

gasAndGasPrice.sol

etherAndWei.sol 1 X

dataLocation

```

1 //Bhagyes Patil D20A 03
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5
6     uint public oneWei = 1 wei;
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    bool public isOneEther = 1 ether == 1e18;
11
12    // 1. Create oneGwei and set it to 1 gwei
13    uint public oneGwei = 1 gwei;
14
15    // 2. Compare 1 gwei to 10^9 (10**9)
16    bool public isOneGwei = 1 gwei == 10**9;
17 }

```

Explain contract

AI copilot

• Tutorial no.18

LEARNETH

Tutorials list

Syllabus

8.3 Bata Structures - Structs

19 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator assign it s new value (lines 39 and 45).

Watch a video tutorial on structs.

★ Assignment

Create a function `atom` that takes a `size` as a penenator a delete a struct member with the given index in the `season` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Compile

itesol

viability.sol

iFUse.sol

loops.col

arrays.sol

mappings.se

```

35
36
37 // update sent
38 function update(uint _index, string memory _text) public {
39     todo storage text = todos[_index];
40     todo.text = _text;
41 }
42
43 // update completed
44 function toggleComplete((uint _index) public {
45     todo storage todo = todos[_index];
46     todo.completed = (todo.completed);
47 }
48 //Bhagyes Patil D20A 03
49 function toggleCompleted((uint _index) public {
50     todo storage- todo = todos[_index];
51     todo.completed = (todo.competed)
52 }
53
54 // update remove
55 function remove((uint _index) public {
56     delete todos[ _index];
57 }

```

Explain contract

0

Uitten on all transactions

Filter with transaction hash

Type the library name to see available commands.

- Tutorial no.19

The screenshot displays the LEARNETH IDE interface. On the left, a sidebar shows the 'Tutorials list' with '8.3 Data Structures - Structs' selected. Below this, the 'Accessing structs' and 'Updating structs' sections are visible, along with an 'Assignment' section that prompts the user to create a function. The main area on the right shows a Solidity code editor with the following code:

```
35
36
37 // update sent
38 function update(uint _index, string memory _text) public {
39     todo.storage.text = todos[_index];
40     todo.text = _text;
41 }
42
43 // update completed
44 function toggleComplete(uint _index) public {
45     todo.storage.todo = todos[_index];
46     todo.completed = (todo.completed);
47 }
48 //Bhagyesh Patil D28A 03
49 function toggleCompleted(uint _index) public {
50     todo.storage.todo = todos[_index];
51     todo.completed = (todo.completed);
52 }
53
54 // update remove
55 function remove(uint _index) public {
56     delete todos[_index];
57 }
```

At the bottom, there is a 'Filter with transaction hash' input field and a 'Type the library name to see available commands.' prompt.

Conclusion:-

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT NO.5

Name:Bhagyesh Patil

Class:D20A

Roll No:03

Batch:A

Aim:Deploying a Voting/Ballot Smart Contract

Theory:-

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping:**

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal

they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage:**

In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type**

Converter help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

- **Code:**

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    // This declares a new complex type which will
    // be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted; // if true, that person already voted
        address delegate; // person delegated to
        uint vote; // index of the voted proposal
    }

    // This is a type for a single proposal.
    struct Proposal
    {
        string name; // Changed from bytes32 to
        string uint voteCount;
    }
    address public chairperson;

    // This declares a state variable that
    // stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) { // Changed bytes32[] to string[]
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        for (uint i = 0; i < proposalNames.length; i++)
            { proposals.push(Proposal{
```

```

    }
  }
}

```

```

name:
proposalNames[i],
voteCount:
0

```

```

/**
 * @dev Give 'voter' the right to vote on this ballot. May only be called by
 'chairperson'.
 * @param voter address of voter
 */
function giveRightToVote(address voter) external {
  // If the first argument of `require` evaluates
  // to 'false', execution terminates and all
  // changes to the state and to Ether balances
  // are reverted.
  // This used to consume all gas in old EVM versions, but
  // not anymore.
  // It is often a good idea to use 'require' to check if
  // functions are called correctly.
  // As a second argument, you can also provide an
  // explanation about what went wrong.
  require(
    msg.sender == chairperson,
    "Only chairperson can give right to vote."
  );
  require(
    !voters[voter].voted,
    "The voter already voted."
  );
  require(voters[voter].weight == 0, "Voter already has the right to vote.");
}

```

```

    voters[voter].weight = 1;
}

/**
 * @dev Delegate your vote to the voter 'to'.
 * @param to address to which vote is delegated
 */
function delegate(address to) external {
    // assigns reference
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");

    require(to != msg.sender, "Self-delegation is disallowed.");

    // Forward the delegation as long as
    // 'to' also delegated.
    // In general, such loops are very dangerous,
    // because if they run too long, they might
    // need more gas than is available in a block.
    // In this case, the delegation will not be executed,
    // but in other situations, such loops might
    // cause a contract to get "stuck" completely.

```



```

while (voters[to].delegate != address(0)) {
    to = voters[to].delegate;

    // We found a loop in the delegation, not allowed.
    require(to != msg.sender, "Found loop in delegation.");
}

Voter storage delegate_ = voters[to];

// Voters cannot delegate to accounts that cannot vote.
require(delegate_.weight >= 1);

// Since 'sender' is a reference, this
// modifies
'voters[msg.sender]'.
sender.voted = true;
sender.delegate = to;

if (delegate_.voted) {
    // If the delegate already voted,
    // directly add to the number of votes
    proposals[delegate_.vote].voteCount +=
    sender.weight;
} else {
    // If the delegate did not vote yet,
    // add to her weight.
    delegate_.weight +=
    sender.weight;
}
}

/**
 * @dev Give your vote (including votes delegated to you) to proposal
'proposals[proposal].name'.
 * @param proposal index of proposal in the proposals array
 */
function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");
    sender.voted = true;
    sender.vote = proposal;

    // If 'proposal' is out of the range of the array,
    // this will throw automatically and revert all
    // changes.
    proposals[proposal].voteCount += sender.weight;
}

/**
 * @dev Computes the winning proposal taking all previous votes into account.
 * @return winningProposal_ index of winning proposal in the proposals array
 */
function winningProposal() public view
    returns (uint
        winningProposal_)
{
    uint winningVoteCount = 0;

```

```

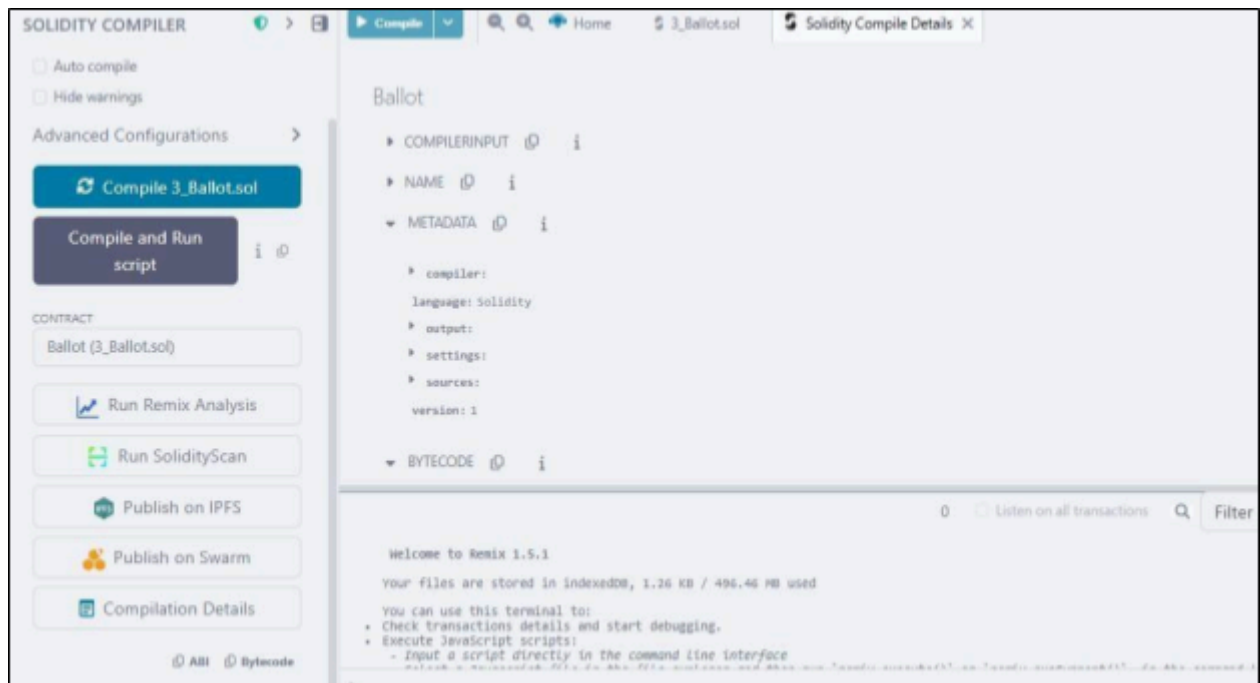
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    /**
     * @dev Calls winningProposal() function to get the index of the winner
    contained in the proposals array and then
     * @return winnerName_ the name of the winner
    */
    function winnerName() external view
        returns (string memory
            winnerName_)
    {
        winnerName_ = proposals[winningProposal()].name;
    }
}

```

Output:-

1)Compile the Ballot.sol Contract



2) Deploying and running of the contract

DEPLOY & RUN TRANSACTIONS

0x5B3...eddC4 (99.999999999999999999)

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

Ballot - contracts/3_Ballot.sol

even version: make

Deploy [Aryan Patankar, "Vishnu", ""]

At Address Load contract from Address

Transactions recorded 1

Deployed Contracts 1

BALLOT AT 0XD91...39138 (ME)

```
154 }
155 }
156 }
157 }
158 /**
159  * @dev Calls winningProposal() function to get the index of the winner contained in the propo
160  * @return winnerName_ the name of the winner
161  */
162 //Bhagyesh Patil D2BA 01
163 function winnerName() external view { infinite gas
164     returns (string memory winnerName_)
165 }
166 {
167     winnerName_ = proposals[winningProposal()].name;
```

Explain contract

Type the library name to see available commands.

creation of Ballot pending...

[v0] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x00...0000 logs: 0 hash: 0x278...b12c5

status 1 transaction mined and execution succeed

transaction hash 0x2784c329447f080c5640f3556c5d0cc4a8040505a480260920184012c5

block hash 0x08030813f28a79a7380015430108f9f0bca377901f4f040505030a7e132a77a

block number 1

contract address 0xd9145c0cc1d08a7254917e481e44a60343f29138

DEPLOY & RUN TRANSACTIONS

0x5B3...eddC4 (99.999999999999999999)

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

Ballot - contracts/3_Ballot.sol

even version: make

Deploy [Aryan Patankar, "Vishnu", ""]

At Address Load contract from Address

Transactions recorded 1

Deployed Contracts 1

BALLOT AT 0XD91...39138 (ME)

```
142 /**
143  * @dev Computes the winning proposal taking all previous votes into account.
144  * @return winningProposal_ index of winning proposal in the proposals array
145  */
146 function winningProposal() public view { infinite gas
147     returns (uint winningProposal_)
148 }
149 {
150     uint winningVoteCount = 0;
151     for (uint p = 0; p < proposals.length; p++) {
152         if (proposals[p].voteCount > winningVoteCount) {
153             winningVoteCount = proposals[p].voteCount;
154             winningProposal_ = p;
155         }
156     }
```

Explain contract

decoded input { "string[] proposalNames": ["Aryan Patankar", "Vishnu", "Nora"] }

decoded output -

logs {}

new logs {}

4) Viewing the details of the Proposal Candidate

✓ BALLOT AT 0XD91...39138 (ME)

Balance: 0 ETH

delegate

address to

▼

giveRightToVote

address voter

▼

vote

uint256 proposal

▼

chairperson

0: address: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

proposals

proposals - call

▼

0: string: name Aryan Patankar

1: uint256: voteCount 0

voters

address

▼

winnerName

winningProposal

Low level interactions

i

CALLDATA

Transact

5) Giving the right to an account other than and by the chairman

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active, displaying the 'giveRightToVote' function. The 'address to' field is set to '0xab8483f64d9c6015cf9b849ae677d033'. The 'uint256 proposal' field is set to '0'. The 'address' field is set to '0x58380da6a701c568545dcfc803fc8875f56beddC4'. The 'string: name Aryan Patankar' and 'uint256: voteCount 0' are also visible. The main editor shows the Solidity code for the 'winningProposal' function. The right sidebar shows the 'Explain contract' panel with transaction details.

```

142  /**
143   * @dev Computes the winning proposal taking all previous votes into account.
144   * @return winningProposal_ index of winning proposal in the proposals array
145   */
146  function winningProposal() public view {
147      returns (uint winningProposal_)
148  }

```

Transaction details:

- status: 1 Transaction mined and execution succeed
- transaction hash: 0xb0c939ce6f8b5336f86bda71f6a691e6b88c8c621958f8bcbf88556a6
- block hash: 0x57dc15c9fa967095463781d88e27c3e069666f031304128506674a59fc05c25
- block number: 2
- from: 0x58380da6a701c568545dcfc803fc8875f56beddC4
- to: Ballot.giveRightToVote(address) 0xd9145cc520380f2549176481a044e9943f39138
- transaction cost: 48641 gas
- execution cost: 27258 gas
- output: 0x
- decoded input: { "address voter": "0xab8483f64d9c6015cf9b849ae677d03315835c2"

6) Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active, displaying the 'vote' function. The 'address' field is set to '0x58380da6a701c568545dcfc803fc8875f56beddC4'. The 'uint256: voteCount 0' is also visible. The main editor shows the Solidity code for the 'winningProposal' function. The right sidebar shows the 'Explain contract' panel with transaction details.

```

142  /**
143   * @dev Computes the winning proposal taking all previous votes into account.
144   * @return winningProposal_ index of winning proposal in the proposals array
145   */
146  function winningProposal() public view {
147      returns (uint winningProposal_)
148  }

```

Transaction details:

- status: 1 Transaction mined and execution succeed
- transaction hash: 0xc0880f3085a04f57447075a3630afafce3a7c276a051c0479628a3a2738
- block hash: 0x0889a0c130130a280a50a589a0870577206a5c32a50075090f29f8790a0
- block number: 1
- from: 0xab8483f64d9c6015cf9b849ae677d03315835c2
- to: Ballot.vote(uint256) 0xd9145cc520380f2549176481a044e9943f39138
- transaction cost: 72584 gas

7) We can view the Voter's Information

The screenshot displays a web interface for a voting system. On the left, there's a sidebar with a 'voters' tab selected. Below it, a list of voters is shown, including 'uint256 weight 1', 'is boot voted true', 'address: delegate 0x00', and 'uint256 vote 0'. There are buttons for 'winningProposal' and 'winningProposal'. Below these, a 'Low level interactions' section shows 'CALLDATA' and a 'Transaction' button. On the right, a transaction details panel is visible, showing a transaction to 'Ballot.vote pending ...'. The transaction details include a green status icon, a transaction hash, block hash, block number, from address, to address, transaction cost, and execution cost.

8) Selecting the account of the delegator and then writing the address of the voter to be delegated to.

The screenshot shows a Solidity IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active. It shows the 'Remix VM (Cancel)' environment. The 'ACCOUNT' section displays the address '0xAb6...35cb2 (99.999999999999748988 I)'. The 'GAS LIMIT' is set to 'Estimated Gas' with a value of '3000000'. The 'VALUE' is set to '0 Wei'. The 'CONTRACT' section shows 'Ballot - contracts/3_Ballot.sol'. There are buttons for 'Deploy', 'At Address', and 'Load contract from Address'. On the right, the 'Solidity Compiler Details' panel shows the source code for the 'winningProposal' function. The function is a public view function that takes an address and returns the index of the winning proposal. Below the code, the 'Explain contract' panel shows the transaction details, including the block number, from address, to address, transaction cost, execution cost, output, decoded input, decoded output, logs, and raw logs.

DEPLOY & RUN TRANSACTIONS

CONTRACT
Ballot - contracts/3_Ballot.sol
own version: osaka

Deploy
["Aryan Patankar", "Vaishnav", "Ning"]

At Address
Load contract from Address

Transactions recorded 10

Deployed Contracts 1

BALLOT AT 0xD91...39138 (MEMORY)

Balance: 0 ETH

delegate
0xAb843F6A49C8d1EcF96B84A6677d033

giveRightToVote
0x4820963Bc4B1177ec7E8F571ceCaf8A9e22C82db

vote
vote - transact (not payable)

chaisperson

Compile

Home

3_Ballot.sol

Solidity Compile Details

```

142
143  /**
144   * @dev Computes the winning proposal taking all previous votes into account.
145   * @return winningProposal_ index of winning proposal in the proposals array
146   */
147  function winningProposal() public view {
148      infinite gas

```

Explain contract

0

Listen on all transactions

block number

7

from

0x4820963Bc4B1177ec7E8F571ceCaf8A9e22C82db

to

Ballot.delegate(address) 0xd9145CCF52D186F254917e481e844e9943F39138

transaction cost

61199 gas

execution cost

39767 gas

output

0x

decoded input

{
"address to": "0xAb843F6A49C8d1EcF96B84A6677d03315835c32"
}

decoded output

()

logs

[]

raw logs

[]

9) Proposal Candidate's Information before giving the Delegate's vote

proposals

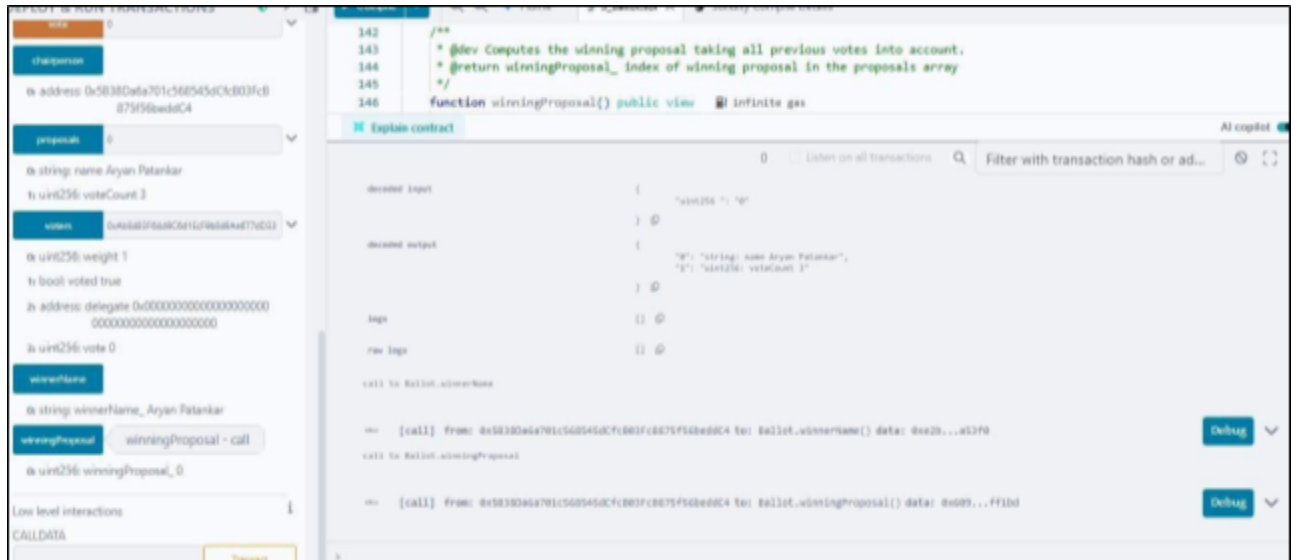
0

0: string: name Bhagyesh Patil

1: uint256: voteCount ()

1: uint256: voteCount 3

10) In this final screenshot we can see that after the delegation's vote the weight of one vote of the one voter who was delegated is the number of people who delegated the voter as their voter (Default weight of any voter is 1)



Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.6

Name:Bhagyesh Patil

Class:D20A

Roll No:3

Batch:A

Aim:Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** The contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Implementation:

The project “**Land Ownership Registry System**” focuses on automating land records and ownership transfers using Ethereum-based Smart Contracts.

To ensure the system works, we will implement two primary contracts:

1. **LandRegistry.sol:** Manages the database of lands and their owners.
2. **LandTransfer.sol:** Handles the logic of transferring ownership from a Seller to a Buyer.

Code:-

1. LandRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract LandRegistry {
    struct Land {
        uint id;
        string location;
        uint area;
        address owner;
        bool isRegistered;
    }

    mapping(uint => Land) public
    lands; address public admin;

    constructor() {
        admin = msg.sender;
    }

    function registerLand(uint _id, string memory _location, uint _area, address _owner)
    public {
        require(msg.sender == admin, "Only admin can register land");
        require(!lands[_id].isRegistered, "Land already registered");

        lands[_id] = Land(_id, _location, _area, _owner, true);
    }

    function updateOwner(uint _id, address _newOwner) public {
        // This function will be called by the Transfer contract
        lands[_id].owner = _newOwner;
    }

    function getLand(uint _id) public view returns (uint, string memory, uint, address) {
        Land memory l = lands[_id];
```

```
        return (l.id, l.location, l.area, l.owner);
    }
}
```

2.LandTransfer.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
import
```

```
"./LandRegistry.sol";
```

```
contract LandTransfer {
```

```
    LandRegistry public registry;
```

```
    constructor(address _registryAddress) {
        registry = LandRegistry(_registryAddress);
    }
```

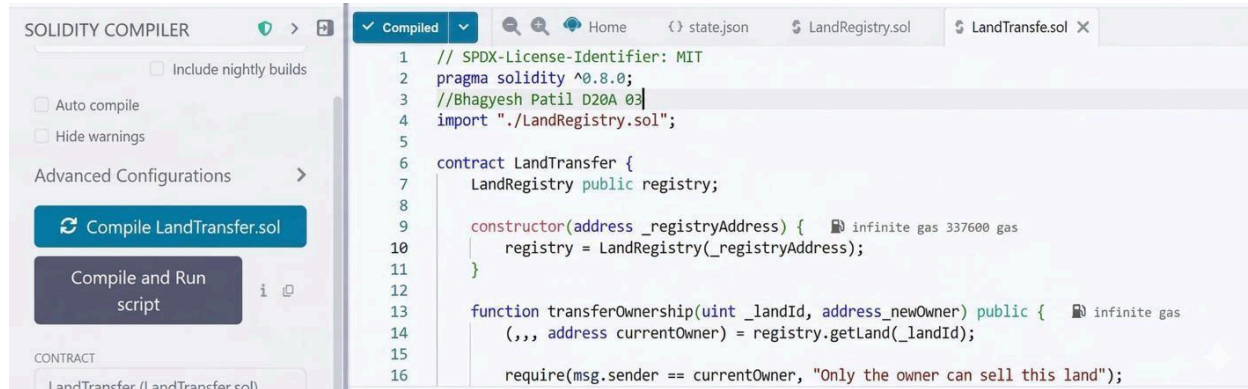
```
    function transferOwnership(uint _landId, address _newOwner) public {
        (,,, address currentOwner) = registry.getLand(_landId);
```

```
        require(msg.sender == currentOwner, "Only the owner can sell this land");
```

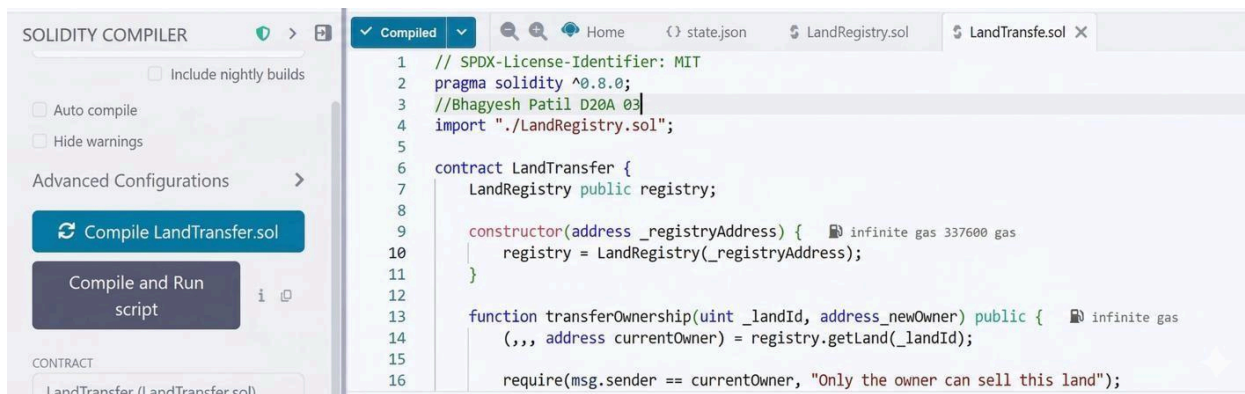
```
        registry.updateOwner(_landId, _newOwner);
    }
}
```

Step 1: Set up the Files

1. Go to Remix IDE.
2. In the File Explorer, create a new folder named LandRegistryProject.
3. Create two files: LandRegistry.sol and LandTransfer.sol.



Step 2: Compile both the contracts



Step 3: Deploy the LandRegistry contract

The screenshot displays the Remix IDE interface during the deployment of the LandRegistry contract. The left sidebar contains the 'TRANSACTIONS' panel with a selected account '0x5B3...eddC4 (99.9999999999)', a '+ Authorize Delegation' button, and gas limit settings (Estimated Gas: 3000000). Below this is the 'CONTRACT' section showing 'LandRegistry - LandRegistryProject/L' and an 'evm version: osaka' tag. The 'Deploy' button is highlighted in orange. The bottom of the sidebar shows 'Transactions recorded' (1) and 'Deployed Contracts' (1), with a link to 'LANDREGISTRY AT 0XC61...96F4'.

The main editor displays the Solidity code for the LandRegistry contract:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3 //Bhagyesh Patil D20A 03
4 import "./LandRegistry.sol";
5
6 contract LandTransfer {
7     LandRegistry public registry;
8
9     constructor(address _registryAddress) {
10         registry = LandRegistry(_registryAddress);
11     }
12
13     function transferOwnership(uint _landId, address _newOwner) public {
14         (,,, address currentOwner) = registry.getLand(_landId);
15         require(msg.sender == currentOwner, "Only the owner can sell this land");
16     }
17 }
```

Below the code is an 'Explain contract' button. The right sidebar shows the transaction details for the deployment:

- Transaction hash: 0xb56f7e987d2d8325540002d1bb3d3cd6c26577addbb830c9d156fa2c4c83ac68
- Block hash: 0x97cd49221691d65cb4aa5e98bbf679bffa4debeea6cf34669b25e943463de3ab
- Block number: 124
- Contract address: 0xc611d78A0287DfdF85fa81c22135977721dd96f4c
- From: 0x5838da6a701c568545dcfc807fc8878f56beddc4

Step 4: Copy the contract address of the deployed LandRegistry contract

15
16

require(msg.sender == currentOwner, "only the owner can sell this land");

✖ Explain contract

0 ☐ Listen on all transactions 🔍

transaction hash	0xb50f7e067d2d9329540023d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60 🔗
block hash	0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab 🔗
block number	124 🔗
contract address	0xC61d78A9287DfdF85fAB1c22135977721dd96F4c 🔗
from	0x5838Da6a701c568545dCfcB03Fc8875f56beddC4 🔗
to	LandRegistry.(constructor) 🔗
transaction cost	824515 gas 🔗

Step 5: Select LandTransfer here in the dropdown

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

LandTransfer - LandRegistryProject/Li

LandRegistry - LandRegistryProject/LandRegistry.sol

LandTransfer - LandRegistryProject/LandTransfer.sol

At Address

Load contract from Address

Transactions recorded 1 >

Deployed Contracts 2

> LANDREGISTRY AT 0XC61...96F4 🔗 ⚙️ ✕

> LANDTRANSFER AT 0XAZE...DCI 🔗 ⚙️ ✕

4 import './LandRegistry.sol';

5

6 contract LandTransfer {

7 LandRegistry public registry;

8

9 constructor(address _registryAddress) { ☐ Infinite gas 337600 gas

10 registry = LandRegistry(_registryAddress);

11 }

12

13 function transferOwnership(uint _landId, address _newOwner) public { ☐ Infinite gas

currentOwner = registry.getLand(_landId);

require(msg.sender == currentOwner, "Only the owner can sell this land");

plain contract

creation of LandRegistry pending...

✓ [vm] from: 0x583...eddC4 to: LandRegistry.(constructor) value: 0 wei data: 0x000...F8053 logs: 0 hash: 0xb50...3ac60

status 1 Transaction mined and execution succeed

transaction hash 0xb50f7e067d2d9329540023d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60 🔗

block hash 0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab 🔗

block number 124 🔗

contract address 0xC61d78A9287DfdF85fAB1c22135977721dd96F4c 🔗

Step 6: Add the contract address in place of `_registryAddress` field next to the deploy button, and click on deploy so as to deploy the LandTransfer contract

DEPLOY & RUN
TRANSACTIONS

+ Authorize Delegation

GAS LIMIT

☒ Estimated Gas

☐ Custom

3000000

VALUE

0

Wei

CONTRACT

LandTransfer - LandRegistryProject/Li

evm version: osaka

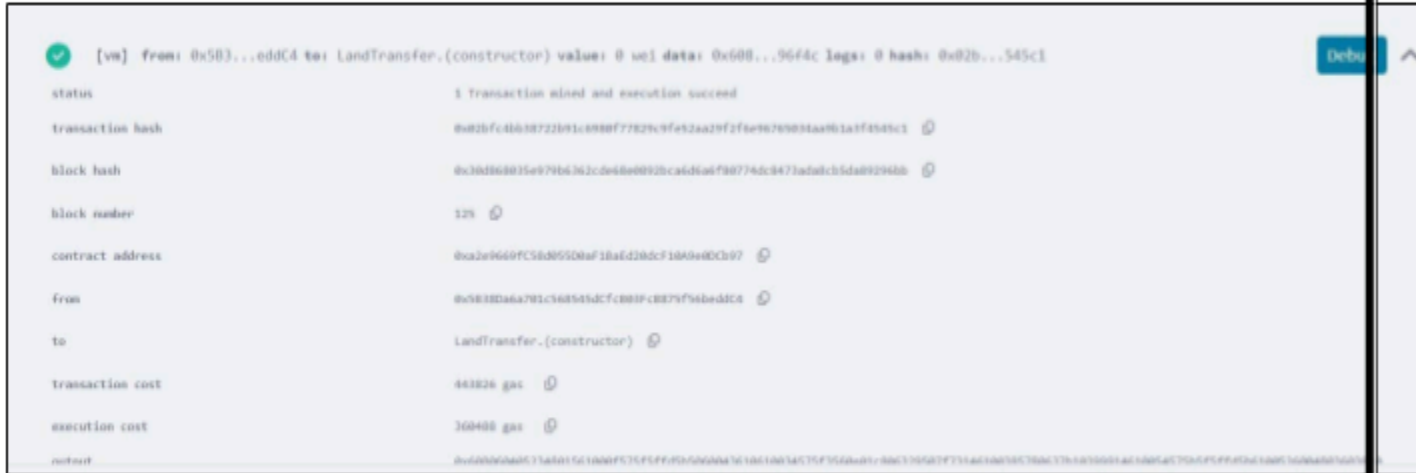
Deploy

0xC61d78A92B7DfdF85fAB1c22

▼

At Address

Load contract from Address



Step 7: Execute the transactions

1.Land Registration

In the deployed contracts section expand the LandRegistry contract, and add details as mentioned. Also ensure that the wallet address is added in the owner field. Once all this is done, click on transact to complete the transaction.

Deployed Contracts 2

LANDREGISTRY AT 0XC61...96F

Balance: 0 ETH

REGISTERLAND

_id: 1

_location: "Mumbai"

_area: 500

_owner: 0x5B38Da6a701c568545dCfcB03Fc1

Calldata

Parameters

transact

DEPLOY & RUN



TRANSACTION

S

Remix VM

(Osaka)

ENVIRONMENT

ACCOU MT -1- If 9 i

C

Reset

Std



0xSB3...eddC4

(99.9999999999

0x5B3...edclC4

(99.99999999999746

0xAb8...35cb2 (100.0 ETH)

0x4B2...C02db (100.0 ETH)

0x787...cabnB (100.0 ETH)

0xG17...5E7f2 (100.0 ETH)

0x17F...8c372 (100.0 ETH)

0x5c6...21678 (100.0 ETH)

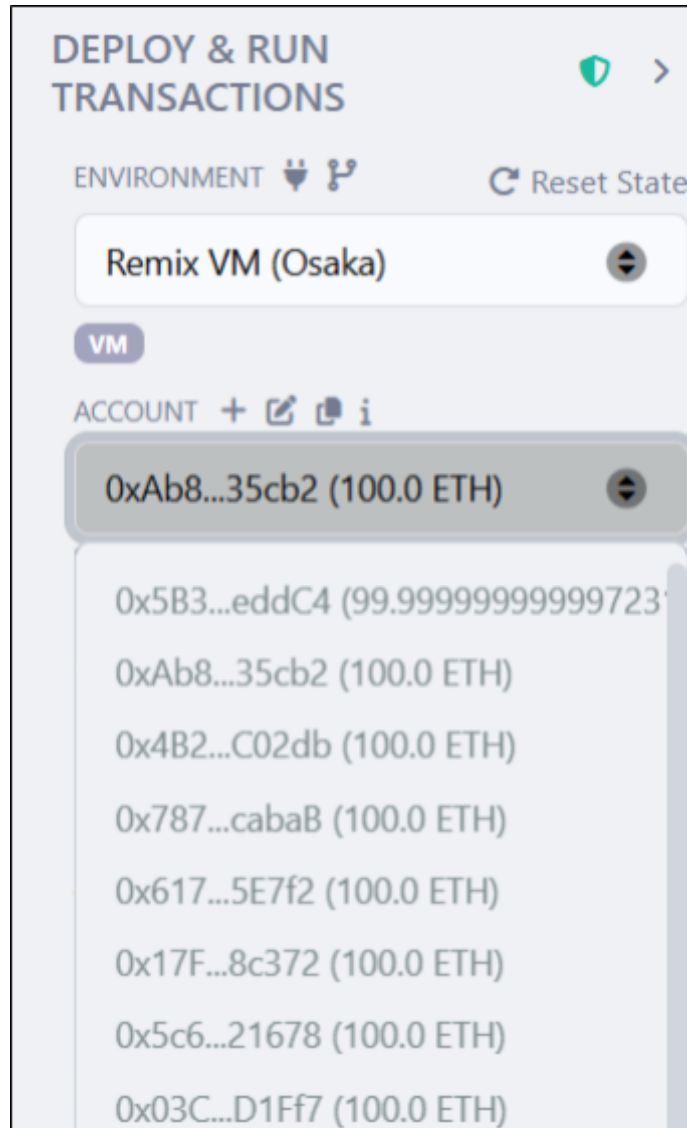
0x03C...D1Ff7 (100.0 ETH)

0x1aE ..E454C (100.0 ETH)

0x0A0...C70DC (100.0 ETH)

2. Transfer Ownership

Copy a different wallet address from the account dropdown (this wallet address represents the buyer)



Switch back to the original wallet address(i.e the seller's wallet address) and within the transferOwnership function in the land transfer contract, enter:-

_landId: **1**

_newOwner: **[The Buyer's Address]**

✓ LANDTRANSFER AT 0XA2E...DC

✕

Balance: 0 ETH

TRANSFEROWNERSHIP

_landId:

1

_newOwner:

0xAb8483F64d9C6d1EcF9b849Ae67

Calldata

Parameters

transact

registry

Click on transact to complete the ownership transfer

0☐ Listen on all transactions

Filter with transaction hash or ad...

transact to LandTransfer.transferOwnership pending ...

✓

[vm] from: 0x583...eddC4 to: LandTransfer.transferOwnership(uint256,address) 0xa2e...DCb97 value: 0 wei data: 0x295...35cb2 logs: 0
hash: 0xc18...d9a2b

Debug

status

1 Transaction mined and execution succeed

transaction hash

0xc188c22aff87cc55a3c65a78a5b87c409e82389c374b8ee4eb3c844867ad9a2b

block hash

0x5c0070a50908a04791208b4e0b0ead28a240afw02d2ba727f8bba0881d796w6d

block number

127

from

0x58380a6a793c5685450cf803f80875f34bwd0C4

to

LandTransfer.transferOwnership(uint256,address) 0xa2e9609fC58d9f508af38afC20dcf386be0C367

transaction cost

43957 gas

execution cost

22385 gas

output

0x

decoded input

{
 "uint256 landId": "1"
}

Now, verify the ownership transfer by going to the LandRegistry contract. When we click on the getLand button, we see a different wallet address, indicating that the ownership transfer has taken place and is successfully verified.

Deployed Contracts

REGISTERLAND

_id: 1

_location: "Mumbai"

_area: 500

_owner: 0x5838Da6a701c56854dCfc803Fd

Calldata Parameters transact

updateOwner uint256_id, address_newO

admin

getLand 1

0: uint256: 1

1: string: Mumbai

2: uint256: 500

3: address: 0xABB483F64d9C6d1EcF9b849Ae677dD3315835cb2

decode uint256

Home {} state.json LandRegistry.sol x LandTransfer.sol x

```
2 pragma solidity ^0.8.0;
3 //Bhagyesh Patil D20A 03
4 import "./LandRegistry.sol";
5
6 contract LandTransfer {
```

Explain contract

transaction cost 43957 gas

execution cost 22385 gas

output 0x

decoded input { "uint256 _landId": "1", "address _newOwner": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2" }

decoded output {}

logs []

raw logs []

call to LandRegistry.getLand

CALL [call] from: 0x5838Da6a701c56854dCfc803Fc8875f56beddC4 to: LandRegistry.getLand(uint256) data: 0xf02.

Conclusion:-

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Experiment No: 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

Blockchain is a decentralized and distributed digital ledger that records transactions securely across multiple systems. It ensures transparency, security, and immutability, meaning once data is recorded, it cannot be changed.

Ethereum is a popular blockchain platform that allows the execution of Smart Contracts. Smart contracts are self-executing programs written in Solidity that automatically perform actions when predefined conditions are met, eliminating the need for intermediaries.

MetaMask is a cryptocurrency wallet and browser extension used to interact with blockchain networks. It stores user accounts and private keys securely and allows users to send, receive, and sign transactions. It also acts as a bridge between Remix IDE and the blockchain network.

Testnets such as Sepolia and RSK Testnet are used for testing purposes. They provide free tokens, allowing developers to deploy and test smart contracts without using real cryptocurrency, reducing financial risk.

Remix IDE is a web-based development environment used to write, compile, deploy, and test smart contracts. It provides a user-friendly interface and supports direct integration with MetaMask for deploying contracts on blockchain networks.

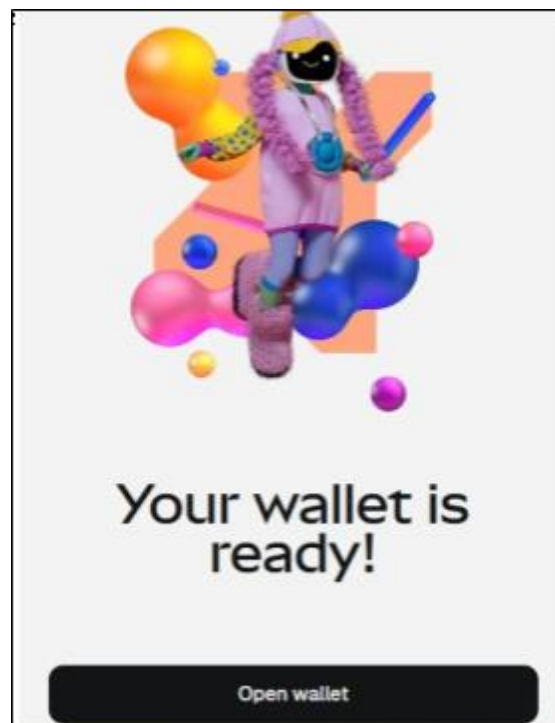
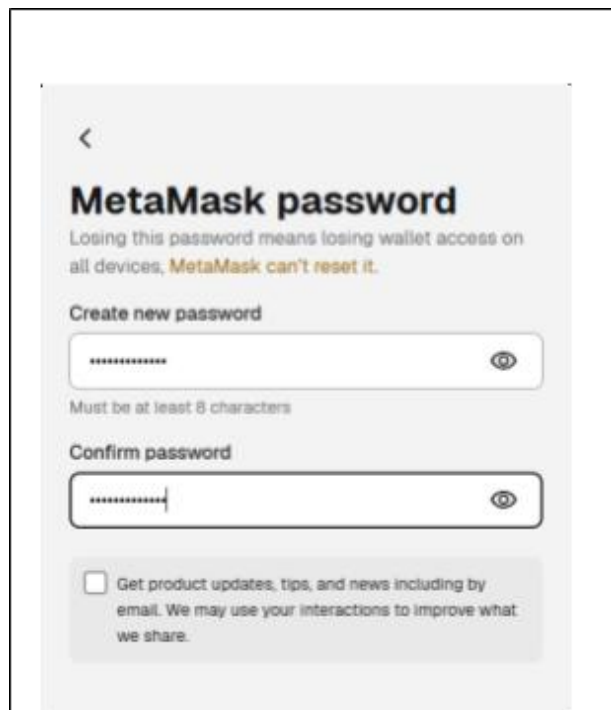
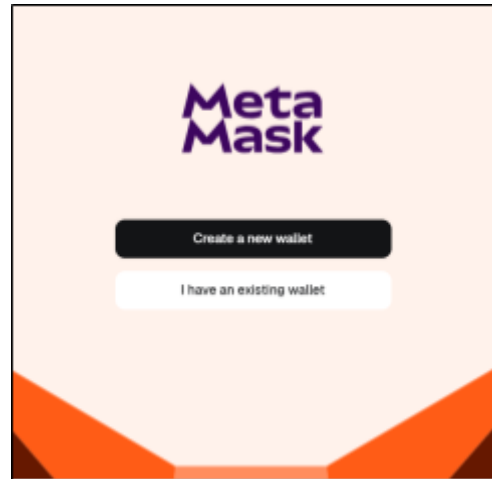
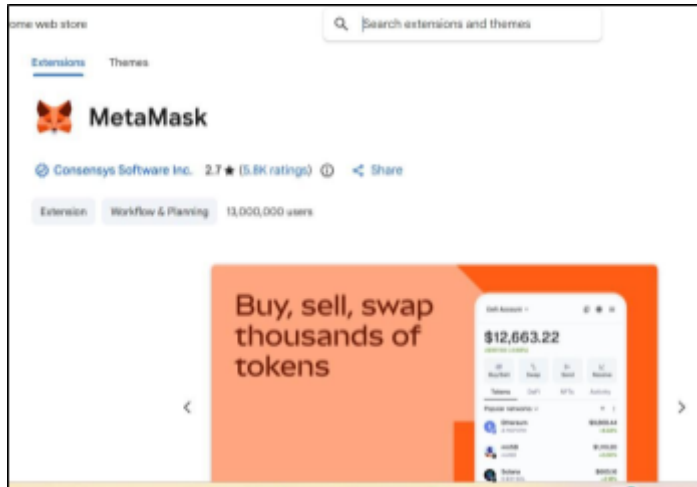
The smart contract deployment process includes writing the contract in Solidity, compiling it to remove errors, connecting MetaMask to Remix, and deploying the contract on a testnet by paying gas fees using test tokens.

A blockchain transaction refers to any interaction with the network, such as deploying a contract or calling its functions. Each transaction requires authentication through a digital signature and is permanently recorded on the blockchain. These transactions can be verified using blockchain explorers.

Implementation

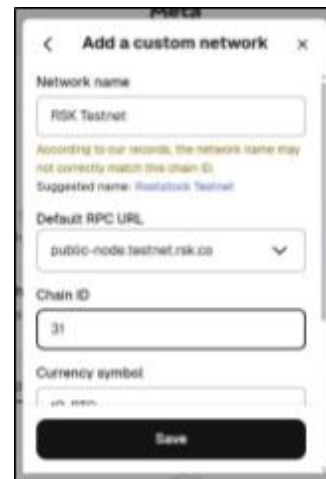
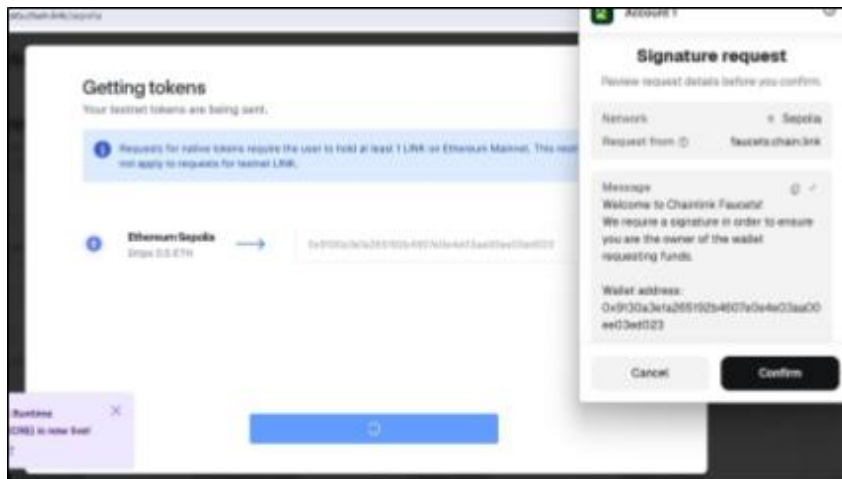
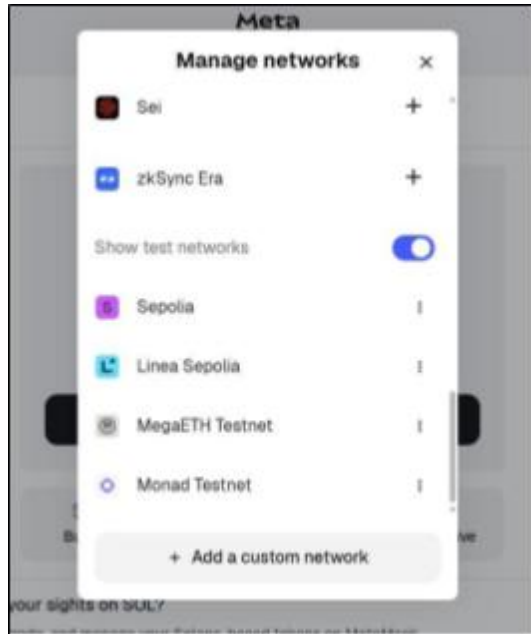
Step 1: Install MetaMask

Install MetaMask extension and create a wallet



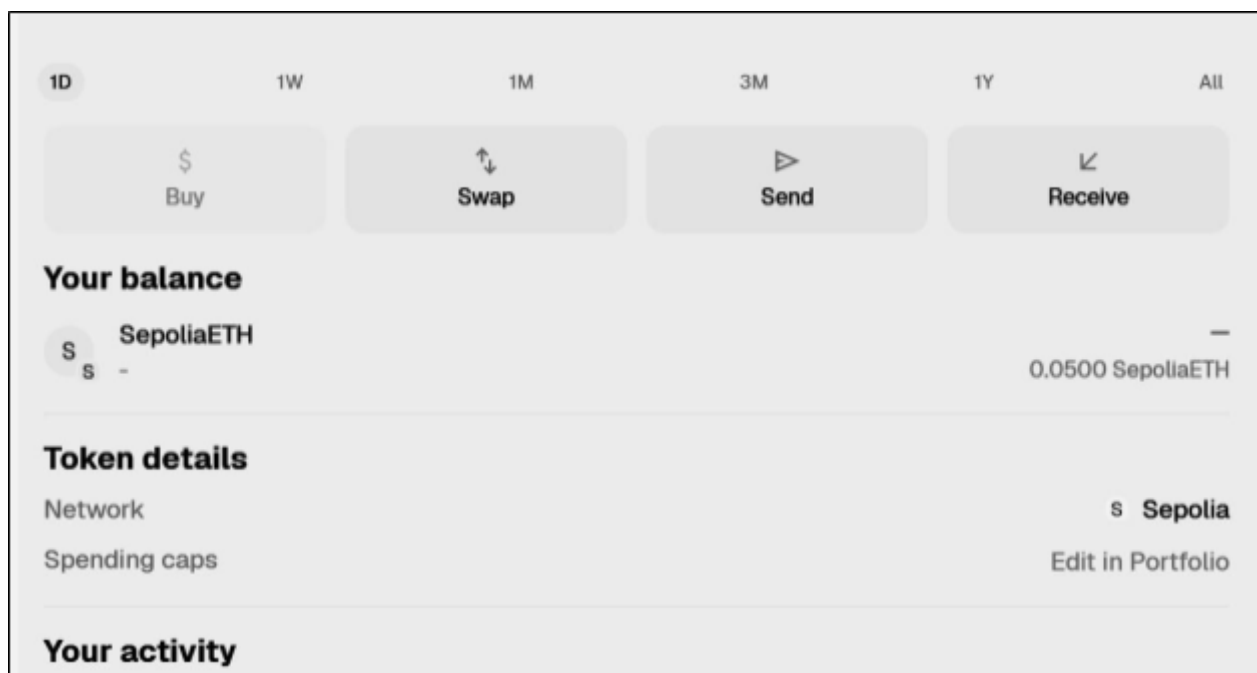
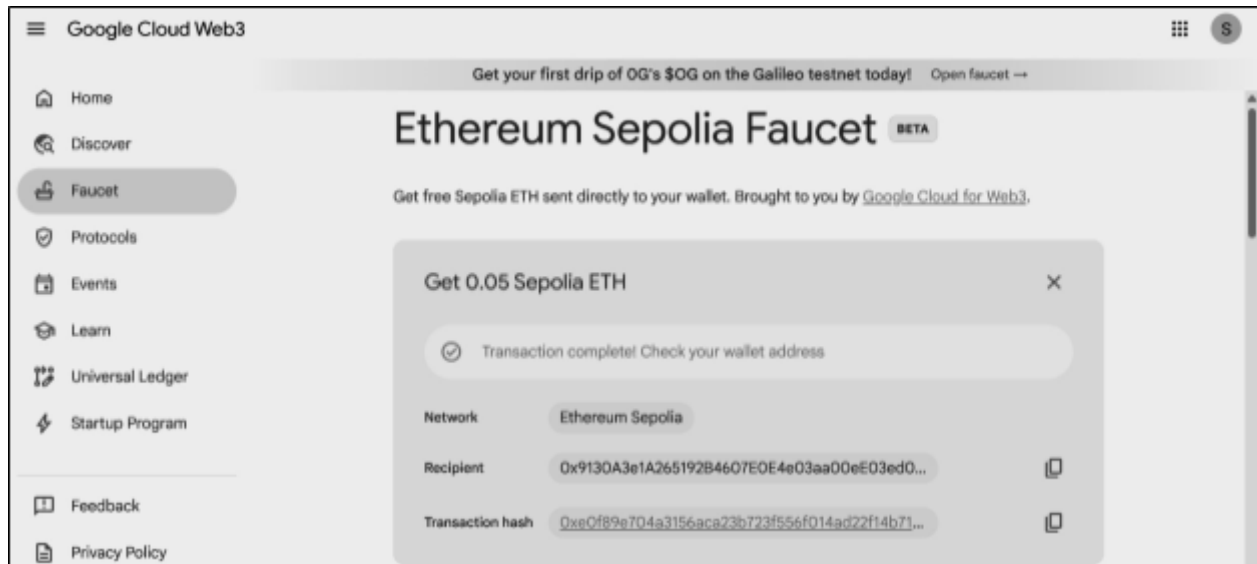
Step 2: Add Sepolia Testnet

Configure MetaMask to connect to Sepolia network.



Step 3: Fund Wallet

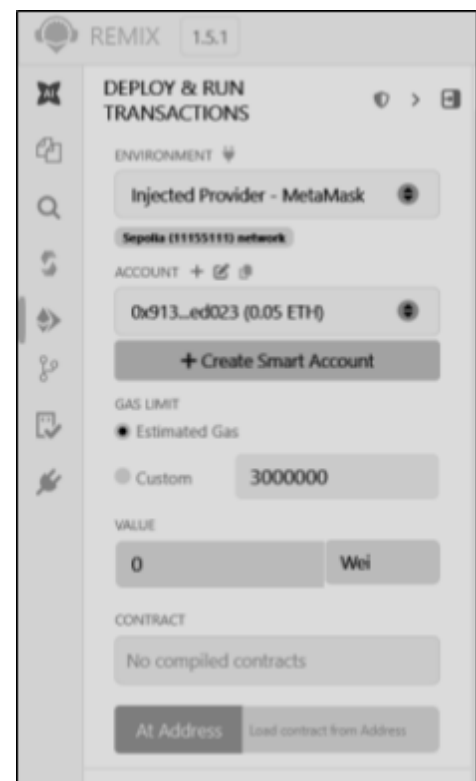
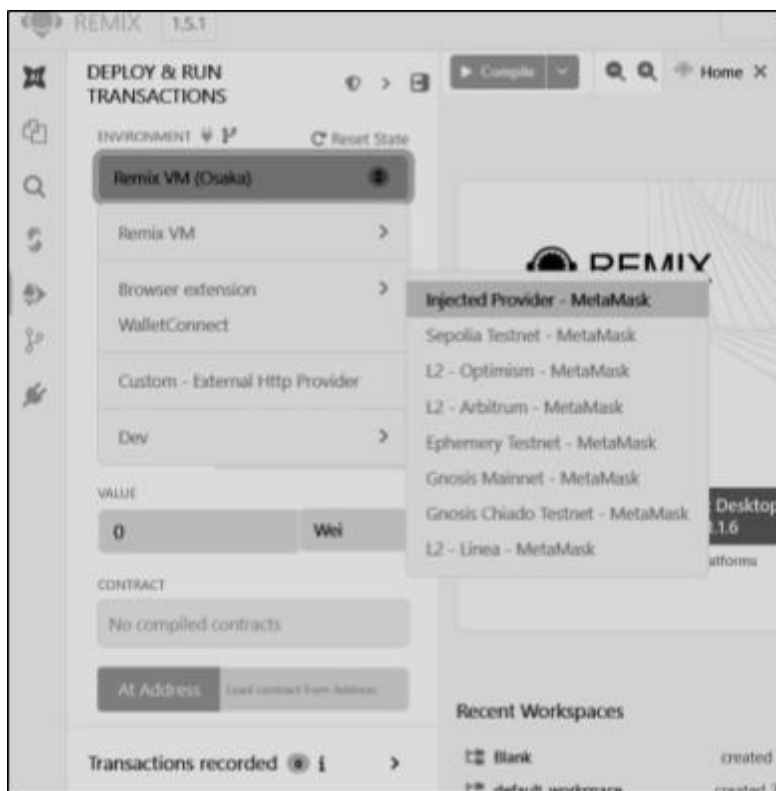
Use faucet to get test ETH



Step4: Connect the Sepolia Testnet / RSK Testnet to Remix IDE

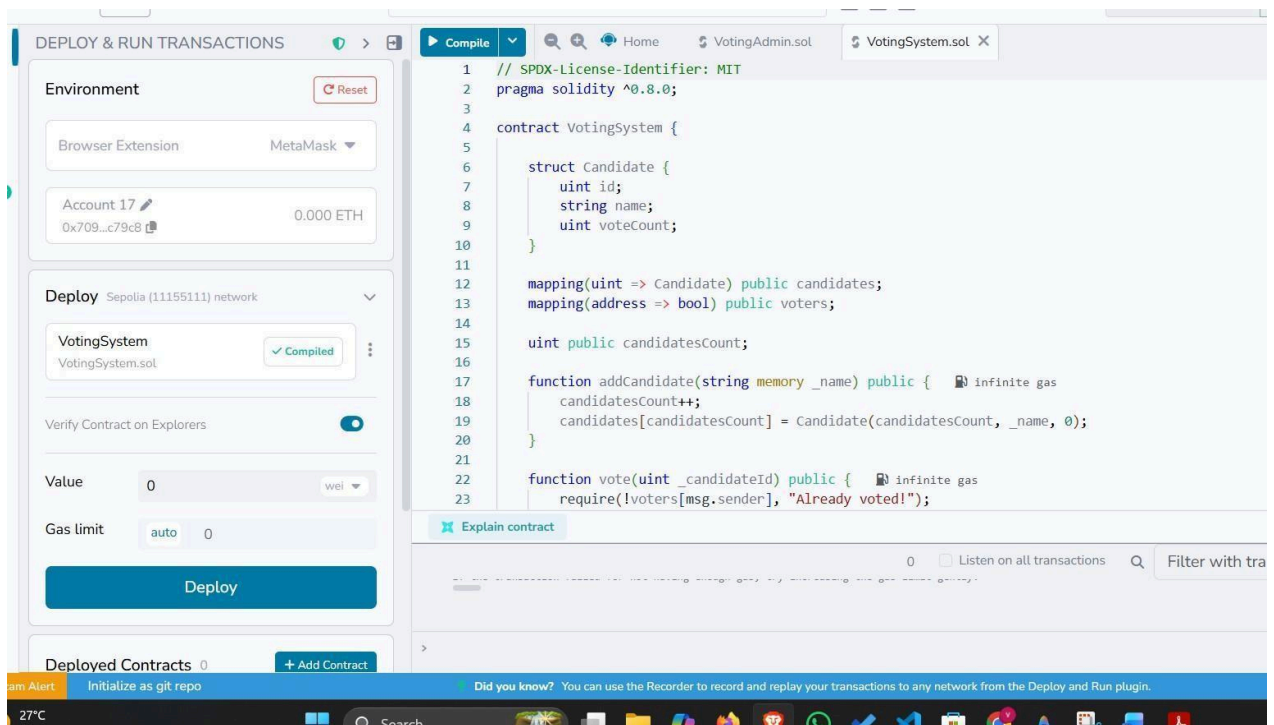
1. In the Remix IDE, select the Environment as Injected Web3.

Note : Injected Web3 connects Remix with the active account in Metamask.



Compile the Contract

1. Open Remix
2. Paste your contract code
3. Go to Solidity Compiler (left panel)
4. Select version 0.8.x (match ^0.8.0)
5. Click Compile Votingsystem.sol



Connect MetaMask (Injected Provider)

1. Go to Deploy & Run Transactions tab
2. In Environment → select Injected Provider - MetaMask
3. MetaMask popup will open →

click Connect Make sure:

- Network = Sepolia Test Network
- You have 0.05 Sepolia ETH

Environment

@' Reset

Browser Extensi... MetaMask w

Account 19 „'
0x862...c1199 L

0.000 ETH

Deploy Sepolia (11155111) network w

VotingSystem

VotingSystem.sol

CompLed



Verify Contract on Explorers



Value

0

wei s'

Gas limit

custom 3o0o0o0

Dep foy

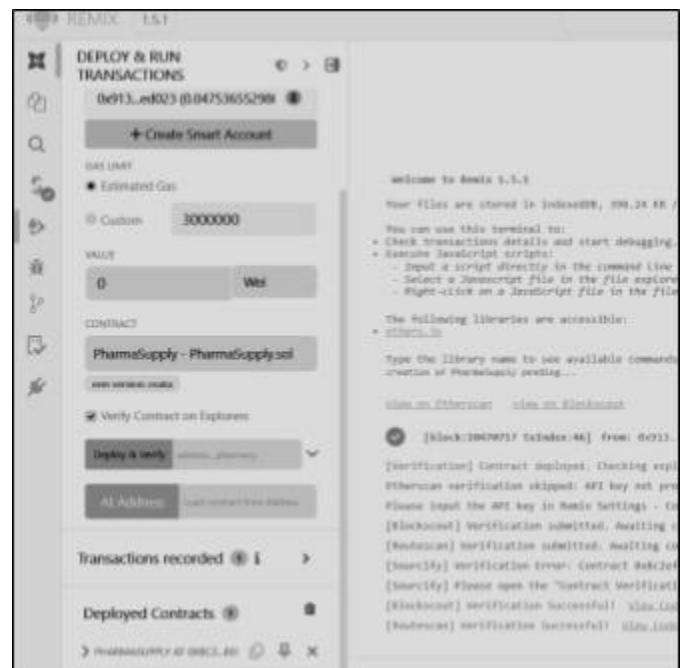
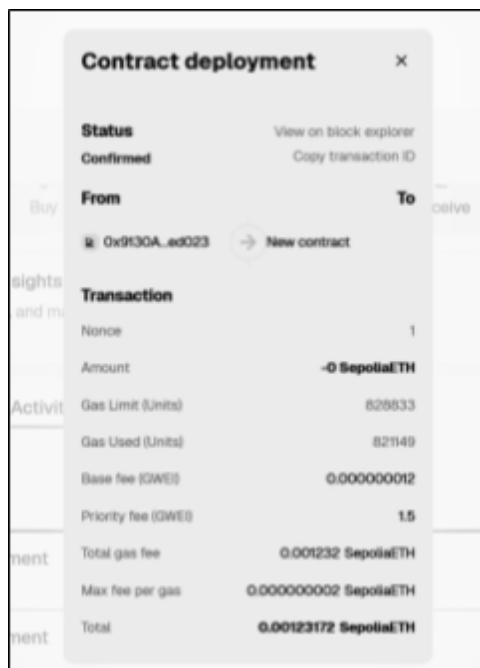
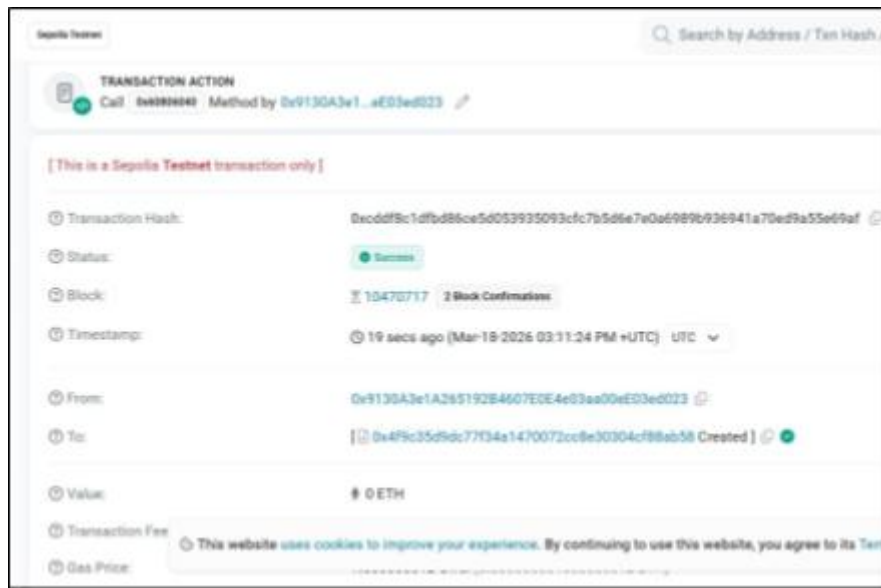
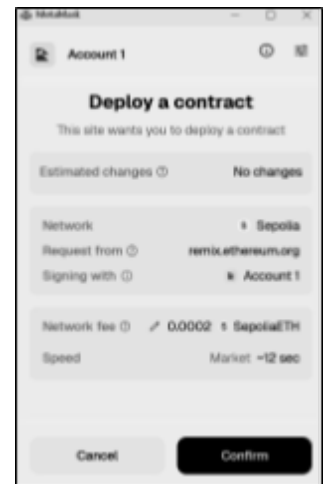
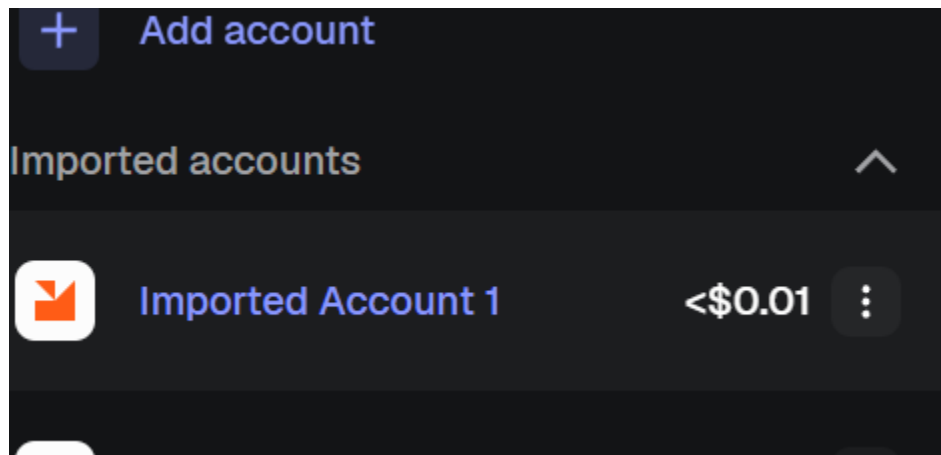
Deploy Contract

Your constructor needs a Voting address Steps:

1. Copy your second MetaMask account address (voting)
 - MetaMask → Account 2 → Copy address
2. Paste that address in constructor field
3. Click Deploy
4. MetaMask will pop up → Confirm

transaction Wait a few seconds...

After success → your contract appears under Deployed Contracts



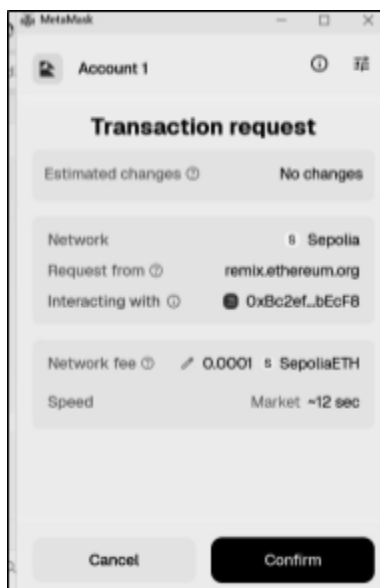
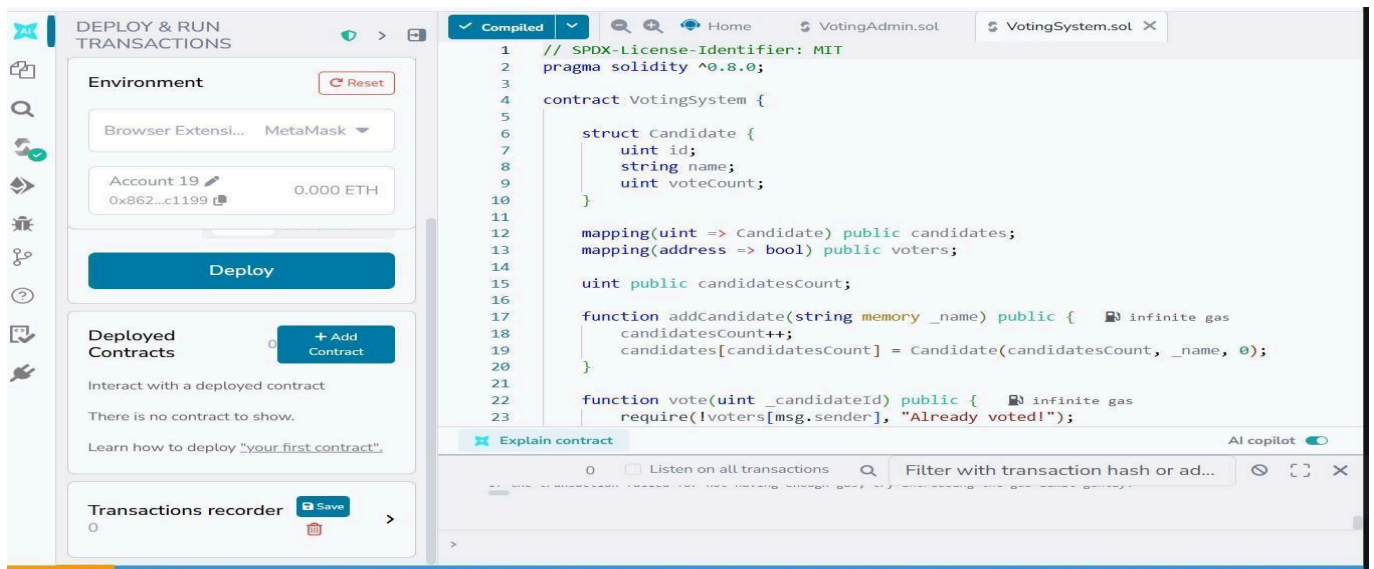
Interact with Contract

Step 1: Add Medicine (Manufacturer only)

- Make sure MetaMask is on Account 1 (manufacturer)
- In Remix:
 - Enter:
 - `_candidate: "Alice"`
- Click

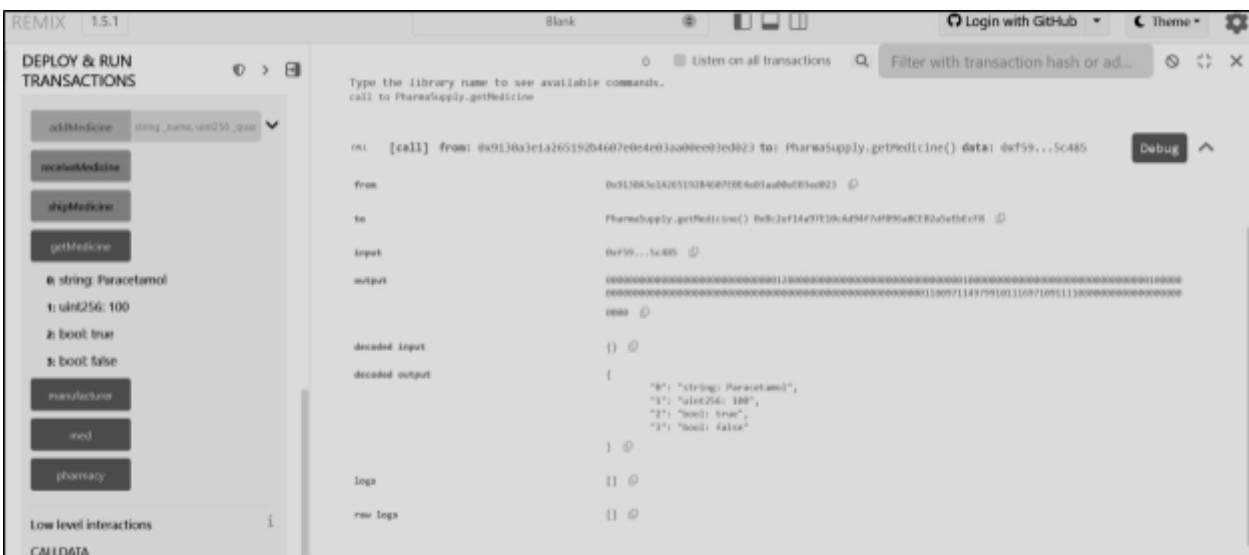
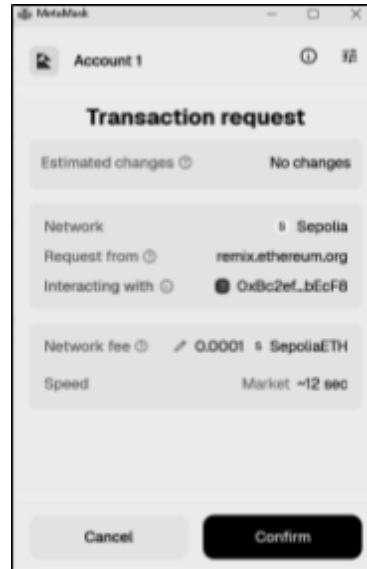
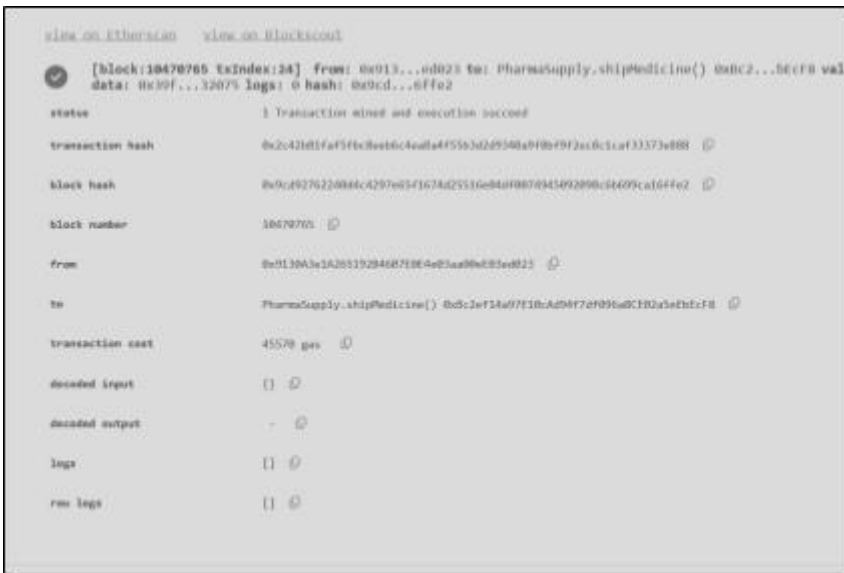
addCandidate Confirm

in MetaMask



Step 2: Calculate Vote

- Still on Account 1
- Click calculateVote
- Confirm transaction



Conclusion

The experiment was successfully carried out by integrating the MetaMask wallet with Remix IDE and performing transactions on the Sepolia test network. The smart contract was compiled, deployed, and interacted with using MetaMask, which enabled secure transaction signing and real-time confirmation on the blockchain.

The use of a test network allowed safe execution without real financial risk, while also providing practical exposure to gas fees, transaction validation, and blockchain explorers. This experiment helped in understanding wallet integration, smart contract deployment, and the overall working of decentralized applications.

Thus, the objective of implementing MetaMask and performing blockchain transactions was successfully achieved.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Experiment 8

AIM: Implementing a Blockchain Platform using Ganache

Theory:

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

- Decentralization: No single entity controls the network; authority is distributed across nodes.
- Immutability: Once data is written to the blockchain, it cannot be altered or deleted.
- Transparency: All transactions are visible to network participants.
- Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).
- Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM), which executes the contract logic on every node in the network.

Key concepts:

- Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.
- ABI (Application Binary Interface): Defines how to interact with the deployed contract — the function signatures and data encodings.
- Address: Every deployed contract gets a unique Ethereum address on the blockchain.
- msg.sender: A global variable in Solidity that holds the address of the account that called the current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates

a full Ethereum node locally on your machine, providing:

- 10 pre-funded test accounts, each loaded with 100 ETH.
- Instant mining — transactions are confirmed immediately (no wait time).
- A local RPC server (default: <http://127.0.0.1:7545>) that tools like MetaMask and Remix can connect to.
- A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data.

Implementation:

Step 1: Install Ganache

Ganache is a personal blockchain for Ethereum development used to deploy contracts, develop applications, and run tests.

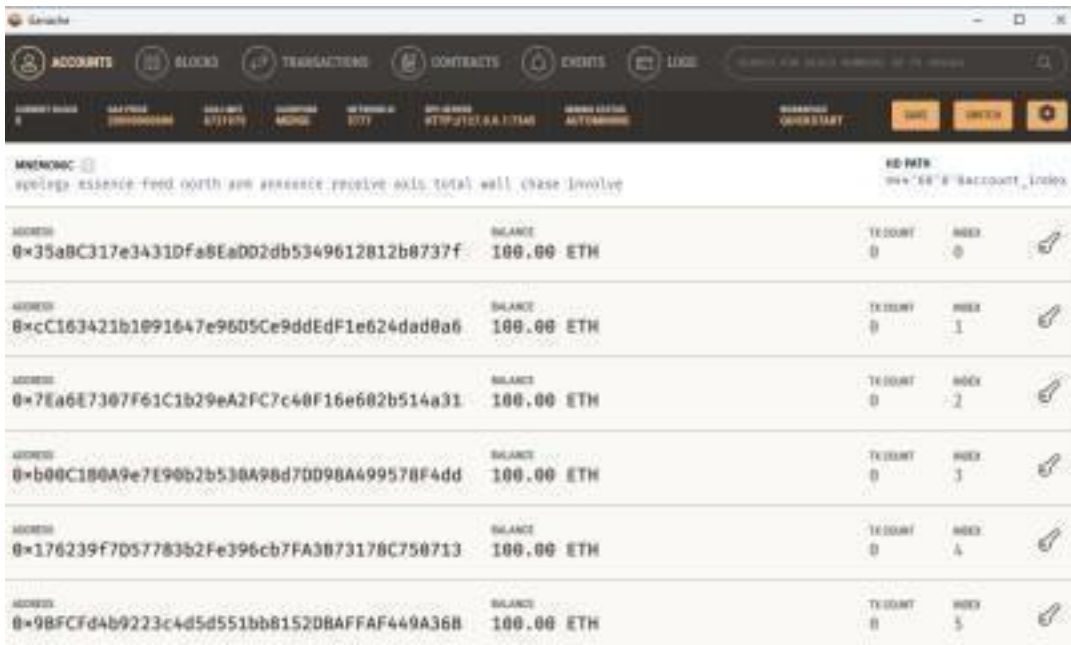
1. Download Ganache v2.7.1 from the official Truffle Suite website ([release assets.githubusercontent.com](https://github.com/trufflesuite/ganache/releases)).

2. Run the installer. The installation prompt shows publisher as Consensys Software Inc.
3. Check 'Launch when ready' and click Install.



4. Once installed, open Ganache and click Quickstart Ethereum to start a local blockchain.





Note the following details from Ganache for later use:

- RPC Server URL: <http://127.0.0.1:7545>
- Network ID: 5777
- Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications.

2.1 Install MetaMask

5. Open Google Chrome and go to the Chrome Web Store.
6. Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.



7. After installation, click 'Create a new wallet' on the MetaMask welcome screen.

8. Set a password and save the Secret Recovery Phrase securely.



2.2 Add Ganache Network to MetaMask

9. In MetaMask, click the network dropdown and select 'Add a custom network'.

10. Enter the following details for the

RPC URL: RPC URL:

`http://127.0.0.1:7545`

RPC Name: Ganache

11. Click 'Add URL'.



12. Fill in the remaining custom network details:

- Network Name: Localhost 8545

- Default RPC URL: Ganache (127.0.0.1:7545)
- Chain ID: 1337
- Currency Symbol: ETH

13. Click Save.

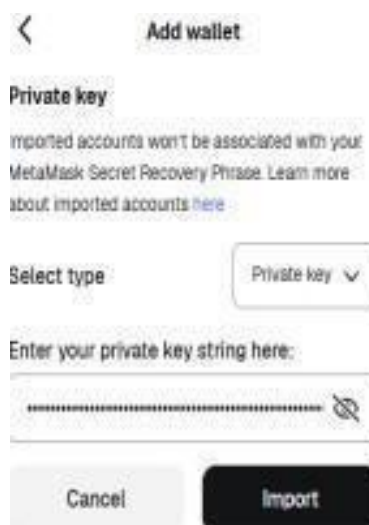


The screenshot shows the 'Add a custom network' interface in MetaMask. It includes the following fields and values:

- Network name:** Localhost 8545
- Default RPC URL:** Ganache (127.0.0.1:7545)
- Chain ID:** 1337
- Currency symbol:** ETH
- Block explorer URL:** (empty)
- Action:** A black 'Save' button at the bottom.

2.3 Import Ganache Accounts into MetaMask

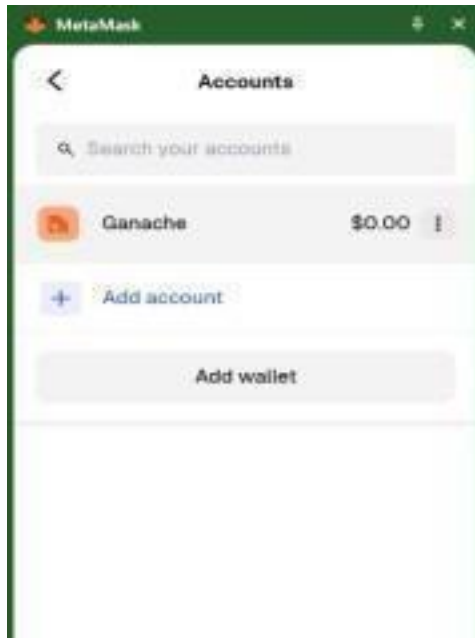
14. In Ganache, click the key icon next to an account address to reveal its private key.
15. In MetaMask, go to Accounts > Add wallet > Import using Private Key.
16. Paste the private key and click Import.



The screenshot shows the 'Add wallet' interface in MetaMask. It includes the following elements:

- Section:** Private key
- Warning:** Imported accounts won't be associated with your MetaMask Secret Recovery Phrase. [Learn more about imported accounts here](#)
- Select type:** A dropdown menu currently set to 'Private key'.
- Input:** A text field labeled 'Enter your private key string here:' with a masked input (dots) and a copy icon.
- Buttons:** 'Cancel' and 'Import' buttons at the bottom.

17. The imported account (labeled 'Ganache') now appears in MetaMask under the Localhost 8545 network.



Step 3: Create the Smart Contract in Remix IDE

Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

18. Open remix.ethereum.org in the browser.
19. In the File Explorer, create a new file named VotingSystem.sol.
20. Write the following smart contract for a Voting System

chain: 5777

Step 4: Compile the Smart Contract

21. In Remix, click the Solidity Compiler icon on the left sidebar.
22. Select compiler version 0.8.x.
23. Click Compile SupplyChain.sol.
24. A green checkmark confirms successful compilation.



Step 5: Connect Remix IDE with MetaMask

25. In Remix, navigate to the Deploy & Run Transactions tab.
26. In the Environment dropdown, select Injected Provider - MetaMask.
27. MetaMask will prompt a connection request — click Connect.
28. Ensure MetaMask is switched to the Localhost 8545 (Ganache) network.
29. The imported Ganache account will now appear as the active account in Remix.

Step 6: Deploy the Smart Contract

30. In the Deploy & Run panel, confirm VoteSystem is selected under Contract.
31. Click Deploy. MetaMask will open a confirmation popup.



32. Review the Network fee and click Confirm in MetaMask.

The screenshot shows the MetaMask interface with the 'ACCOUNTS' tab selected. The account name is 'MADMOIRAC' and the balance is 99.92 ETH. Below the account details, there is a list of transactions. The first transaction is highlighted, showing a balance of 99.92 ETH and a transaction count of 2. The other transactions show a balance of 100.00 ETH and transaction counts of 0, 0, 0, and 0 respectively.

ADDRESS	BALANCE	TX COUNT	NAME
0x35aBC317e3431Dfa8EaD02db5349612812b0737f	99.92 ETH	2	
0xc163421b1091647e96D5Ce9ddEdF1e624dad8a5	100.00 ETH	0	1
0x7Ea6E7307F61C1b29eA2FC7c40F16e602b514a31	100.00 ETH	0	2
0xb0C180A9e7E90b2b530A98d7D098A499578F4dd	100.00 ETH	0	3
0x176239f7057783b2Fe396cb7FA3B73178C750713	100.00 ETH	0	4
0x9BFCF4b9223c4d5d551bb815208AFFAF449A368	100.00 ETH	0	5

The contract deployment details in MetaMask show:

- Status: Confirmed
- From: 0x35aBC...0737f → To: New contract
- Gas Used: 849,377 units
- Total gas fee: 0.016988 ETH
- Account balance after: 99.92 ETH

The screenshot shows the 'Contract deployment' dialog box. It displays the status as 'Confirmed' and provides a link to 'View on block explorer'. The 'From' field shows the address 0x35aBC...0737f and the 'To' field shows 'New contract'. The 'Transaction' section lists various fees: Gas Limit (849377), Gas Used (849377), Base fee (0.00000002 ETH), Priority fee (0.00000002 ETH), Total gas fee (0.016988 ETH), Max fee per gas (0.00000002 ETH), and Total (0.016988 ETH).

Field	Value
Status	Confirmed
From	0x35aBC...0737f
To	New contract
Gas Limit (units)	849377
Gas Used (units)	849377
Base fee (GWEI)	0.00000002
Priority fee (GWEI)	0.00000002
Total gas fee	0.016988 ETH
Max fee per gas	0.00000002 ETH
Total	0.016988 ETH

Step 8: Interact with the Smart Contract

After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

Transaction Flow — Pharma Supply Chain

Account 1 (Manufacturer) — Call addMedicine:

```
addMedicine(1, "Paracetamol", 100)
```

Account 2 (Distributor) — Switch account in MetaMask, then call:

```
sendToDistributor(1)
```

Account 3 (Retailer) — Switch account in MetaMask, then call:

```
sendToRetailer(1)
```

Account 4 (Customer) — Switch account in MetaMask, then call:

```
purchase(1)
```

Call getMedicine to verify the full supply chain:

```
getMedicine(1)
```

Expected output from getMedicine(1):

- id: 1
- name: Paracetamol
- quantity: 100
- manufacturer: 0x35aBC...0737f (Account 1 address)
- distributor: 0xcC163...d0a6 (Account 2 address)
- retailer: 0x7Ea6E...a31 (Account 3 address)
- customer: 0xb00C1...4dd (Account 4 address)

Output

- Medicine batch created successfully on the Ganache local blockchain.
- Complete supply chain flow executed: Manufacturer → Distributor → Retailer → Customer.
- All transactions confirmed and visible in Ganache with updated block counts and reduced ETH balances.
- Smart contract deployed with status Confirmed; gas fees deducted correctly.

Conclusion

The experiment was successfully carried out by implementing the Ganache platform and integrating it with MetaMask and Remix IDE. The smart contract was compiled, deployed, and executed using Ganache accounts, which provided a local blockchain environment with pre-funded accounts.

The transactions were processed instantly, and all details such as gas usage, block creation, and account balances were observed directly in Ganache. This provided a clear understanding of how blockchain transactions work in a controlled environment without requiring real cryptocurrency.

Thus, the experiment helped in understanding the practical implementation of a local blockchain, wallet integration, and smart contract deployment successfully.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT NO.9

Name: Bhagyesh Patil

Class:D20A

Roll No:3

Batch:A

Aim: To implement a Private Ethereum Blockchain using Geth.

Theory:

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (DApps)**. Unlike public blockchains such as the Ethereum mainnet, a **Private Ethereum Blockchain** is restricted to specific participants, making it ideal for **development, testing, and enterprise applications** where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through **JSON-RPC, command-line, or JavaScript consoles**.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**

4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
5. **Creating a Genesis Block:**
6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The `geth init` command initializes the blockchain using the `genesis.json` file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**
12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Note: Download the genesis file and edit the account details, including the public keys of all participating peers in the network.

Step 1: Installing Geth on Ubuntu

sudo apt update

```
ubuntu@ubuntu:~/Desktop$ sudo apt update
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Get:4 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease [17.5 kB]
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:6 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [1,519 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [436 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1,303 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [233 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [1,616 kB]
]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [271 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [599 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [852 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [163 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [602 kB]
Get:16 http://in.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [293 kB]
Get:17 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [1,644 kB]
]
Get:18 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [274 kB]
Get:19 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,060 kB]
Get:20 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe i386 Packages [698 kB]
Get:21 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [241 kB]
```

Sudo dpkg --configure -a

```
ubuntu@ubuntu:~/Desktop$ sudo dpkg --configure -a
```

sudo apt install software-properties-common

```
ubuntu@ubuntu:~/Desktop$ sudo apt install software-properties-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-software-properties software-properties-gtk ubuntu-advantage-tools
  ubuntu-pro-client
Recommended packages:
  ubuntu-pro-client-l10n
The following NEW packages will be installed:
  ubuntu-pro-client
The following packages will be upgraded:
  python3-software-properties software-properties-common software-properties-gtk
  ubuntu-advantage-tools
4 upgraded, 1 newly installed, 0 to remove and 231 not upgraded.
Need to get 322 kB of archives.
After this operation, 1,339 kB disk space will be freed.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 ubuntu-advantage-tools a
11 31.2-22.04 [10.8 kB]
```

sudo add-apt-repository -y ppa:ethereum/ethereum

```
ubuntu@ubuntu:~/Desktop$ sudo add-apt-repository -y ppa:ethereum/ethereum
Repository: 'deb https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu/ jammy main'
More info: https://launchpad.net/~ethereum/+archive/ubuntu/ethereum
Adding repository.
Found existing deb entry in /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Adding deb entry to /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Found existing deb-src entry in /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Adding disabled deb-src entry to /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.lis
t
Adding key to /etc/apt/trusted.gpg.d/ethereum-ubuntu-ethereum.gpg with fingerprint 2A518C81
9BE37D2C2031944D1C52189C923F6CA9
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease
Reading package lists... Done
```

sudo apt update

```
ubuntu@ubuntu:~/Desktop$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
231 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ubuntu:~/Desktop$ █
```

sudo apt install geth

```
ubuntu@ubuntu:~/Desktop$ sudo apt install geth
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
geth is already the newest version (1.13.14+build29502+jammy).
geth set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 231 not upgraded.
```

Step 2: Check the version of Geth on the Terminal

geth version

```
ubuntu@ubuntu:~/Desktop$ geth version
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=
GOROOT=
ubuntu@ubuntu:~/Desktop$
```

3. Step - 3: Create a Private Ethereum Network

1. Create a folder named, private_ethereum_setup

```
ubuntu@ubuntu:~/Desktop$ mkdir private_ethereum_setup
```

2. Create 2 subfolders named node1 and node2 in the folder private_ethereum_setup

```
ubuntu@ubuntu:~/Desktop$ cd private_ethereum_setup/
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ mkdir node1 node2
```

3. Create 2 accounts in the folder corresponding to node1 and node2

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 account new
INFO [03-31|10:20:39.436] Maximum peer count          ETH=50 total=50
INFO [03-31|10:20:39.438] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.
comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:  0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
Path of the secret key file: node1/keystore/UTC--2024-03-31T04-50-51.743374765Z--9f9497c56b548
90fd2f693bac26c71e4736eb2be

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ S
```


4. Create a genesis.json file in the folder, `private_etheruem_setup` nano genesis.json

```
ubuntu@ubuntu: ~/Desktop/private_ethereum_setup
```

```
GNU nano 6.2 genesis.json *
```

```
"config": {  
  "chainId": 1234,  
  "homesteadBlock": 0,  
  "eip150Block": 0,  
  "eip155Block": 0,  
  "eip158Block": 0,  
  "byzantiumBlock": 0,  
  "constantinopleBlock": 0,  
  "petersburgBlock": 0,  
  "istanbulBlock": 0,  
  "ethash": {}  
},  
"nonce": "0x0",  
"timestamp": "0x5f5a0a92",  
"extraData": "0x0000000000000000000000000000000000000000000000000000000000000000",  
"gasLimit": "0x8000000",  
"difficulty": "0x4000",  
"mixhash": "0x0000000000000000000000000000000000000000000000000000000000000000",  
"coinbase": "0x0000000000000000000000000000000000000000000000000000000000000000",  
"alloc": {  
  "0x9F9497c56b54890fd2F693BaC26C71e4736eb2bE": {  
    "balance": "100000000000000000000000000000000"  
  },  
  "0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433": {  
    "balance": "100000000000000000000000000000000"  
  }  
}
```

5. Initialize the nodes with the genesis file

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 init genesis.json
INFO [03-31|10:33:21.842] Maximum peer count                      ETH=50 total=50
INFO [03-31|10:33:21.843] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.
comm: no such file or directory"
INFO [03-31|10:33:21.847] Set global gas cap                      cap=50,000,000
INFO [03-31|10:33:21.850] Initializing the KZG library             backend=gokzg
INFO [03-31|10:33:21.919] Defaulting to pebble as the backing database
INFO [03-31|10:33:21.919] Allocated cache and file handles        database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/chaindata cache=16.00MiB handles=16
INFO [03-31|10:33:21.942] Opened ancient database                 database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/chaindata/ancient/chain readonly=false
INFO [03-31|10:33:21.943] State schema set to default             scheme=hash
INFO [03-31|10:33:21.943] Writing custom genesis block
INFO [03-31|10:33:21.945] Persisted trie from memory database      nodes=4 size=480.00B time="
609.399µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:21.953] Successfully wrote genesis state         database=chaindata hash=c63
8fa..fe1846
INFO [03-31|10:33:21.953] Defaulting to pebble as the backing database
INFO [03-31|10:33:21.953] Allocated cache and file handles        database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/lightchaindata cache=16.00MiB handles=16
INFO [03-31|10:33:21.962] Opened ancient database                 database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/lightchaindata/ancient/chain readonly=false
INFO [03-31|10:33:21.962] State schema set to default             scheme=hash
INFO [03-31|10:33:21.963] Writing custom genesis block
INFO [03-31|10:33:21.964] Persisted trie from memory database      nodes=4 size=480.00B time="
811.122µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:21.969] Successfully wrote genesis state         database=lightchaindata has
h=c638fa..fe1846
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ S
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 init genesis.json
INFO [03-31|10:33:56.477] Maximum peer count                      ETH=50 total=50
INFO [03-31|10:33:56.479] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.
comm: no such file or directory"
INFO [03-31|10:33:56.485] Set global gas cap                      cap=50,000,000
INFO [03-31|10:33:56.486] Initializing the KZG library             backend=gokzg
INFO [03-31|10:33:56.571] Defaulting to pebble as the backing database
INFO [03-31|10:33:56.571] Allocated cache and file handles        database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/chaindata cache=16.00MiB handles=16
INFO [03-31|10:33:56.583] Opened ancient database                 database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|10:33:56.583] State schema set to default             scheme=hash
INFO [03-31|10:33:56.583] Writing custom genesis block
INFO [03-31|10:33:56.584] Persisted trie from memory database      nodes=4 size=480.00B time="
695.266µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:56.590] Successfully wrote genesis state         database=chaindata hash=c63
8fa..fe1846
INFO [03-31|10:33:56.590] Defaulting to pebble as the backing database
INFO [03-31|10:33:56.590] Allocated cache and file handles        database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/lightchaindata cache=16.00MiB handles=16
INFO [03-31|10:33:56.602] Opened ancient database                 database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/lightchaindata/ancient/chain readonly=false
INFO [03-31|10:33:56.602] State schema set to default             scheme=hash
INFO [03-31|10:33:56.602] Writing custom genesis block
INFO [03-31|10:33:56.603] Persisted trie from memory database      nodes=4 size=480.00B time="
549.872µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:56.609] Successfully wrote genesis state         database=lightchaindata has
h=c638fa..fe1846
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

6. For configuring the boot node

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ sudo apt-get install bootnode
[sudo] password for ubuntu:
Sorry, try again.
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bootnode is already the newest version (1.13.14+build29502+jammy).
bootnode set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 231 not upgraded.
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -genkey boot.key
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -nodekey boot.key
enode://fbbb379450b32a560c1adbc01f89cd7bf7434c64d3d61140e35f01e4dc6872e079e610606a8127a161a0ed
9d7be778636b22552290f7aaabad2e8232ed823eee@127.0.0.1:0?discport=30301
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|10:35:18.634] New local node record          seq=1,711,861,518,633 id=ba
197519c6e384da ip=<nil> udp=0 tcp=0
```

4. Step - 4: Establish a Peer-Peer Connection between the nodes along with the bootnode

1. On the first Terminal, Use boot.key to run the boot node

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -verbosity 5 -nodekey boot.key -addr 127.0.0.1:30305
enode://fbbb379450b32a560c1adbc01f89cd7bf7434c64d3d61140e35f01e4dc6872e079e610606a8127a161a0ed
9d7be778636b22552290f7aaabad2e8232ed823eee@127.0.0.1:0?discport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|10:40:06.489] New local node record          seq=1,711,861,806,489 id=b
197519c6e384da ip=<nil> udp=0 tcp=0
```

2. On the second Terminal, Run Node 1

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 --port 30306 --bootnodes enode://f7aba85ba36
923bffd3438b4c8fde6b1f02b1c23ea0aac825ed7eac38e6230e5cadcf868e73b0e28710f4c9f685ca71a86a4911461637ae9ab2bd852939b
7f@127.0.0.1:0?discport=30305 --networkid 1234 --unlock 0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE --password "
password.txt" --authrpc.port 8551 --mine --miner.etherbase 0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
INFO [03-31|11:25:31.763] Maximum peer count                      ETH=50 LES=0 total=50
INFO [03-31|11:25:31.765] Smartcard socket not found, disabling      err="stat /run/pcscd/pcscd.comm: no such file
r directory"
INFO [03-31|11:25:31.769] Set global gas cap                          cap=50,000,000
INFO [03-31|11:25:31.771] Allocated trie memory caches              clean=154.00MiB dirty=256.00MiB
INFO [03-31|11:25:31.771] Using pebble as the backing database
INFO [03-31|11:25:31.771] Allocated cache and file handles          database=/home/ubuntu/Desktop/private_ethereum
setup/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|11:25:31.785] Opened ancient database                   database=/home/ubuntu/Desktop/private_ethereum
setup/node1/geth/chaindata/ancient/chain readonly=false
INFO [03-31|11:25:31.792] Disk storage enabled for ethash caches    dir=/home/ubuntu/Desktop/private_ethereum_setu
/node1/geth/ethash count=3
INFO [03-31|11:25:31.796] Disk storage enabled for ethash DAGs      dir=/home/ubuntu/.ethash count=2
INFO [03-31|11:25:31.797] Initialising Ethereum protocol            network=1234 dbversion=8
INFO [03-31|11:25:31.797] -----
INFO [03-31|11:25:31.797] Chain ID: 1234 (unknown)
INFO [03-31|11:25:31.797] Consensus: Ethash (proof-of-work)
INFO [03-31|11:25:31.797] Pre-Merge hard forks (block based):
INFO [03-31|11:25:31.797] - Homestead: #0 (https://github.com/ethereum/execution-specs/
lob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|11:25:31.797] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/
lob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [03-31|11:25:31.797] - Spurious Dragon/1 (EIP 155): #0 (https://github.com/ethereum/execution-specs/
lob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [03-31|11:25:31.797] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/
```

ubuntu@ubuntu: ~/Desktop/private_ethereum_setup	ubuntu@ubuntu: ~/Desktop/private_ethereum_setup
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:49.599] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:50.100] << FINDNODE/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:50.100] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:50.601] << FINDNODE/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:50.601] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:51.102] << FINDNODE/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:51.102] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:51.602] << FINDNODE/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:51.602] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:51.625] << PONG/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:51.625] >> PING/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
DEBUG[03-31 11:27:51.625] Revalidated node	b=16 id=1af3a1e4b66753ff checks=2
TRACE[03-31 11:27:52.103] << FINDNODE/v4	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:52.104] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:52.604] << FINDNODE/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:52.604] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:53.105] << FINDNODE/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
TRACE[03-31 11:27:53.105] >> NEIGHBORS/v4	
ll>	id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n

3. On the third Terminal, Run Node 2

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 --port 30307 --bootnodes enode://f7aba85ba369923bffd3438b4c8fde6b1f02b1c23ea0aac825ed7eac38e6230e5cadcf068e73b0e28710f4c9f685ca71a86a4911461637ae9ab2bd852939b77f@127.0.0.1:0?discport=30305 --networkid 1234 --keystore "/node2/keystore" --unlock 0xfc09dce1A80Fe679e5cC09D88B3E99de13864433 --password "/password.txt" console
```

```
INFO [03-31|11:56:49.157] Maximum peer count                      ETH=50 LES=0 total=50
INFO [03-31|11:56:49.159] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file o
r directory"
INFO [03-31|11:56:49.164] Set global gas cap                      cap=50,000,000
INFO [03-31|11:56:49.167] Allocated trie memory caches           clean=154.00MiB dirty=256.00MiB
INFO [03-31|11:56:49.167] Using pebble as the backing database
INFO [03-31|11:56:49.167] Allocated cache and file handles       database=/home/ubuntu/Desktop/private_ethereum_
setup/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|11:56:49.192] Opened ancient database                 database=/home/ubuntu/Desktop/private_ethereum_
setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|11:56:49.202] Disk storage enabled for ethash caches  dir=/home/ubuntu/Desktop/private_ethereum_setup
/node2/geth/ethash count=3
INFO [03-31|11:56:49.202] Disk storage enabled for ethash DAGs    dir=/home/ubuntu/.ethash count=2
INFO [03-31|11:56:49.203] Initialising Ethereum protocol          network=1234 dbversion=8
INFO [03-31|11:56:49.204]
INFO [03-31|11:56:49.204] Chain ID: 1234 (unknown)
INFO [03-31|11:56:49.205] Consensus: Ethash (proof-of-work)
INFO [03-31|11:56:49.205]
INFO [03-31|11:56:49.205] Pre-Merge hard forks (block based):
INFO [03-31|11:56:49.205] - Homestead: #0 (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|11:56:49.205] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/b
```

5. Step - 5 : Exploring the network by attaching JavaScript console to Node 1

1. Fetch network status

```
> net
{
  listening: true,
  peerCount: 1,
  version: "1234",
  getListening: function(callback),
  getPeerCount: function(callback),
  getVersion: function(callback)
}
```

2. To fetch the number of blocks mined

```
> web3.eth.blockNumber
0
> INFO [03-23|12:15:29.133] Looking for peers                      peercount=1
  tried=0 static=0
```

3. To check the balance of the accounts

```
entage=72 elapsed=0M42.708S
> eth.getBalance(eth.accounts[0])
1.000322e+24
> INFO [03-31|12:09:54.727] Generating DAG in progress
```


7. Fetch the details of the peers in the network

```
> admin.peers
[[
  {
    caps: ["eth/66", "eth/67", "eth/68", "snap/1"],
    enode: "enode://dce739f3f2fae54667568d84488b8d055409c862e80d78d653bf086b250ddd431fa6c6342502f9ff8f99129b5ee055679fab9335a9049ec7ae1b13941bb265e@127.0.0.1:30307",
    id: "ebf13aa79411d6e216f71bee7625c7391e7a2049d3853aed09ffb5566981dc27",
    name: "Geth/v1.11.6-stable-ea9e62ca/linux-amd64/go1.20.3",
    network: {
      inbound: false,
      localAddress: "127.0.0.1:36900",
      remoteAddress: "127.0.0.1:30307",
      static: false,
      trusted: false
    },
    protocols: {
      eth: {
        version: 68
      },
      snap: {
        version: 1
      }
    }
  }
]
```

8. Perform Transactions between peers in the network

```
> eth.sendTransaction({from:"0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE",to:"0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433"})
WARN [03-31|12:16:19.599] Caller gas above allowance, capping      requested=114,569,871 cap=50,000,000
INFO [03-31|12:16:19.617] Setting new local account      address=0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
INFO [03-31|12:16:19.619] Submitted transaction          hash=0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f
e4757b479a1b6812debe56f33f from=0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE nonce=0 recipient=0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433 value=0
"0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f"
> INFO [03-31|12:16:27.814] Looking for peers            peercount=1 tried=0 static=0
```

9. Check the balances of sender and receiver 2

```
> eth.getBalance("0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE")
1.000322e+24
```

10. To check the details of the transaction on Node 1 Terminal

```
> eth.getTransaction("0xe181e503b93f95f2e69553091fbdd8459eda0942662c68ead0c5492779cf31c7")
{
  blockHash: null,
  blockNumber: null,
  chainId: "0x4d2",
  from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
  gas: 21000,
  gasPrice: 1000000000,
  hash: "0xe181e503b93f95f2e69553091fbdd8459eda0942662c68ead0c5492779cf31c7",
  input: "0x",
  nonce: 4,
  r: "0xd7bb1c00265f84c46d3c4fbf9ed8228eb791fb0e0303f1d2098989bfed21ecd1",
  s: "0x676eb39f0310ac550aedd7bade8f2988b99e64fcbcb743cf91d2d889298c6d9",
  to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
  transactionIndex: null,
  type: "0x0",
  v: "0x9c7",
  value: 90009000
}
> INFO [03-31|12:19:48.198] Looking for peers            peercount=1 tried=0 static=0
> INFO [03-31|12:19:58.215] Looking for peers            peercount=1 tried=0 static=0
```

11. To check the contents in the Mempool - Transaction Pool

```
> txpool.content
{
  pending: [
    0x9f9497c56b54890fd2f693bac26c71e4736eb2be: {
      0: {
        blockHash: null,
        blockNumber: null,
        chainId: "0x4d2",
        from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
        gas: "0x5208",
        gasPrice: "0x3b9aca00",
        hash: "0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f",
        input: "0x",
        nonce: "0x0",
        r: "0xfe80302530833e03cb38f654004a63d2b812eeab0d15ac0c743c02959e32b17e",
        s: "0x532a68167bf51184d46643bc35d828d941fc9a75a043b4b37010a7409cb41f1c",
        to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
        transactionIndex: null,
        type: "0x0",
        v: "0x9c8",
        value: "0x0"
      },
      1: {
        blockHash: null,
        blockNumber: null,
        chainId: "0x4d2",
        from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
        gas: "0x5208",
        gasPrice: "0x3b9aca00",
        hash: "0xdd43d6b52e779b78d33a4e5c311ce24b71fc7c7434c93464d662ce20f81ec3c",
        input: "0x",
        nonce: "0x1",
        r: "0xa1abb8a85a39d5da37ac14f8275a2eb5034e8182bd4cfa7b3523d631020188f3",
        s: "0xfb96f3fb07299f86289f4a30805b0872a36b091f9c9c65e577de70adb609e11",
        to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
        transactionIndex: null,

```

12. To check the status of the Mempool - Transaction Pool

```
> txpool.status
{
  pending: 5,
  queued: 0
}
> INFO [03-31|12:21:38.389] Looking for peers                peercount=1 tried=0 static=0
```

Conclusion:-

The experiment successfully implemented a **Private Ethereum Blockchain** using **Geth** and **Clique (PoA)** consensus. By initializing a custom **genesis block**, an isolated network was established for zero-cost testing. This setup proved that authorized signers can effectively validate transactions without the computational overhead of public networks, providing a secure environment for enterprise DApp development.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

EXPERIMENT 10

Name-Bhagyesh Patil

Class-D20A

Roll no-03

Aim: Explore the Cryptocurrency

Landscape Theory:

This experiment explores the **cryptocurrency landscape** by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from **three reputable sources** as per the lab requirement:

1. **CoinGecko API** – Provided historical price data, market capitalization, and trading volumes in a structured JSON format, enabling direct processing in Python.
2. **CoinMarketCap** – Offered real-time and historical price charts, circulating supply, and market rank for visual cross-verification.
3. **Crypto.com** – Supplied quick-access market statistics, simplified price trends, and cryptocurrency news updates.

Cryptocurrency Landscape

1. **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:
 - **Store of Value** – Bitcoin, Litecoin.
 - **Utility Tokens** – Chainlink, Ethereum.
 - **Stablecoins** – USDT, USDC.
 - **Governance Tokens** – UNI, AAVE.
3. **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. **Data Analysis Importance** – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.

5. **Advancements** – Growing use of **DeFi platforms**, **NFT ecosystems**, **layer-2 scaling**, and **cross-chain protocols**.

Data Collection Process

The program used the **CoinGecko API** to automate historical data fetching for Chainlink. The retrieved data was processed with **Pandas** to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using **Matplotlib**, a six-panel dashboard was created to display:

1. **Price Trend** – Daily price movement over the selected period.
2. **Price vs Volume** – Correlation between trading activity and price changes.
3. **Daily Returns Distribution** – Statistical spread of daily gains/losses.
4. **Moving Averages** – Technical analysis trend lines for price movement.
5. **Cumulative Returns** – Overall growth performance.
6. **Rolling Volatility** – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered **advancements in blockchain technology**, including:

- **Consensus Mechanisms** – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- **Scalability Solutions** – Layer-2 protocols, sharding, and sidechains to increase throughput.
- **Interoperability Protocols** – Chainlink's **CCIP**, Polkadot, and Cosmos enabling cross-chain data exchange.
- **Privacy Features** – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

Chainlink's **roadmap** highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward **decentralization, interoperability, and security**.

Market Trends & Adoption

- **Growing Merchant Acceptance** – More businesses accepting cryptocurrency payments.
- **Wallet Growth** – Increasing downloads and installations of crypto wallets.
- **Network Activity** – Higher transaction volumes and active addresses, signaling strong adoption.
- **Integration with Traditional Finance** – Partnerships between crypto platforms and fintech/payment providers.
- **Regulatory Developments** – Countries exploring frameworks for safe cryptocurrency adoption.

Utilizing Api for analysis and visualization :

Code:

```
import requests

import matplotlib.pyplot as plt

import pandas as pd

# ----- Fetch Data -----

def get_historical_data(coin_id, days='180', currency='usd'):

    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'

    params = {'vs_currency': currency, 'days': days}

    response = requests.get(url, params=params)

    return response.json()

# ----- Dashboard Function

-----def

chainlink_dashboard(days='180'):

    coin_id = 'chainlink'
```

```

data = get_historical_data(coin_id, days)

# Create DataFrame
prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

df = pd.merge(prices_df, volumes_df,
on='timestamp') df.set_index('timestamp',
inplace=True)

# Calculate extra metrics
df['returns'] = df['price'].pct_change()
df['MA7'] = df['price'].rolling(window=7).mean()
df['MA30'] = df['price'].rolling(window=30).mean()
df['cumulative'] = (1 + df['returns']).cumprod()
df['volatility'] = df['returns'].rolling(window=7).std() *
100

# ----- Plot Dashboard -----
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

# 1. Price Trend
axes[0, 0].plot(df.index, df['price'], color='blue')
axes[0, 0].set_title('Price Trend')
axes[0, 0].set_ylabel('Price (USD)')
axes[0, 0].set_xlabel('Year-Month') # Add
xlabel axes[0, 0].grid(True)

```

2. Price vs Volume

```
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)
```

3. Daily Returns Distribution

```
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7,
color='purple') axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)
```

4. Moving Averages

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1, 1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)
```

5. Cumulative Returns

```
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
```

```

axes[2,
0].set_xlabel('Year-Month')
axes[2, 0].grid(True)

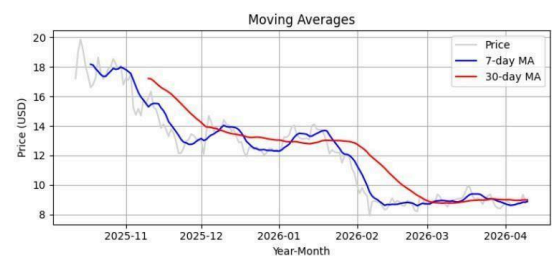
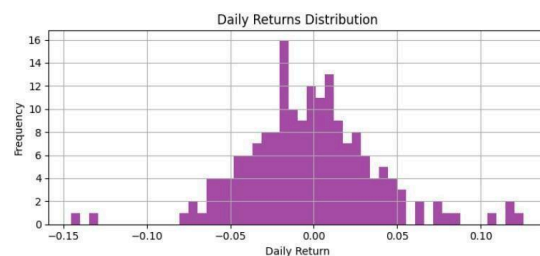
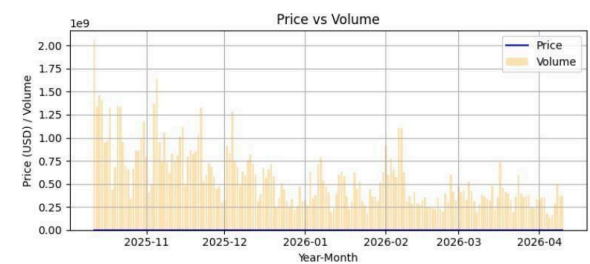
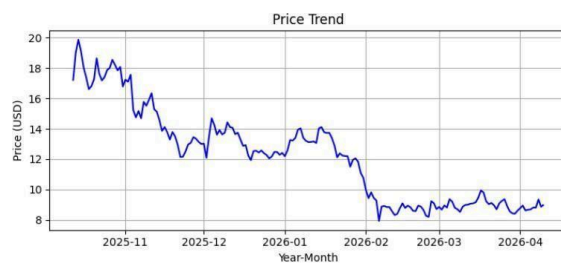
# 6. Volatility
axes[2, 1].plot(df.index, df['volatility'],
color='orange') axes[2, 1].set_title('Rolling Volatility
(7-Day)') axes[2, 1].set_ylabel('Volatility (%)')
axes[2, 1].set_xlabel('Year-Month')
axes[2, 1].grid(True)

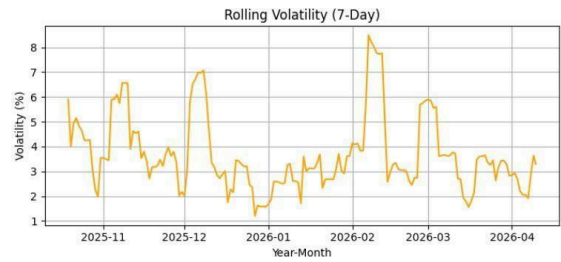
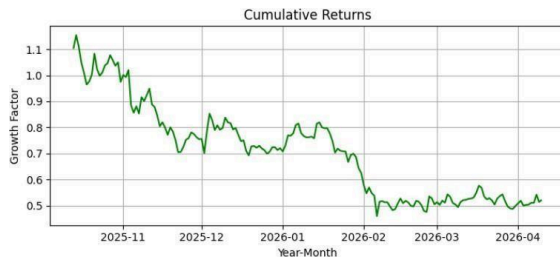
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
plt.show()

# ----- Run Dashboard
-----chainlink_dashboard('180'
)

```

Output:





Coin Gecko Website

coingecko.com

Now LIVE: Spot CEX Report 2026

Coins: 17,728 Exchanges: 1,457 Market Cap: \$2,527T ▲ 0.9% 24h Vol: \$87,101B Dominance: BTC 57.0% ETH 10.6% Gas: 0.126 GWEI

coingecko Cryptocurrencies Exchanges NFT Learn Products API Candy Portfolio Search

GET 5 BTC FOR FREE +100 **PLAY NOW**

Cryptocurrency Prices by Market Cap

The global cryptocurrency market cap today is \$2.53 Trillion, a ▲ 0.9% change in the last 24 hours. [Read more](#)

Metric	Value	Change
Market Cap	\$2,526,881,227,609	▲ 0.9%
24h Trading Volume	\$87,100,760,179	

Trending	Price	Change
Zcash	\$363.04	▲ 11.1%
Bitcoin	\$71,924.45	▲ 1.1%
Siren	\$0.7399	▲ 29.8%

Top Gainers	Price	Change
RaveDAO	\$0.5855	▲ 92.0%
Xphere	\$0.01634	▲ 63.7%
Siren	\$0.7399	▲ 29.8%

coingecko.com/en/coins/bitcoin

Bitcoin BTC Price #1

\$72,037.64 ▲ 1.2% (24h)

1.0000 BTC ▲ 0.0%

24h Range: \$70,522.33 - \$72,495.25

Add to Portfolio • 2,367,265 added

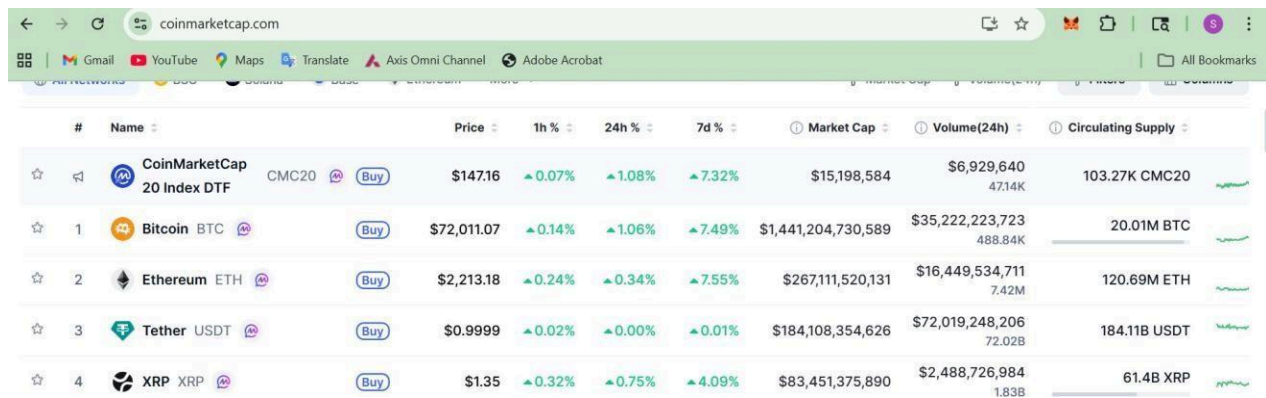
Metric	Value
Market Cap	\$1,441,734,490,650
Fully Diluted Valuation	\$1,441,734,490,650
24 Hour Trading Vol	\$35,707,938,832
Circulating Supply	20,013,637
Total Supply	20,013,637
Max Supply	21,000,000

Overview Markets Treasuries News Similar Coins Historical Data BTC Halving

Price Compare 24H 7D 1M 3M YTD 1Y Max

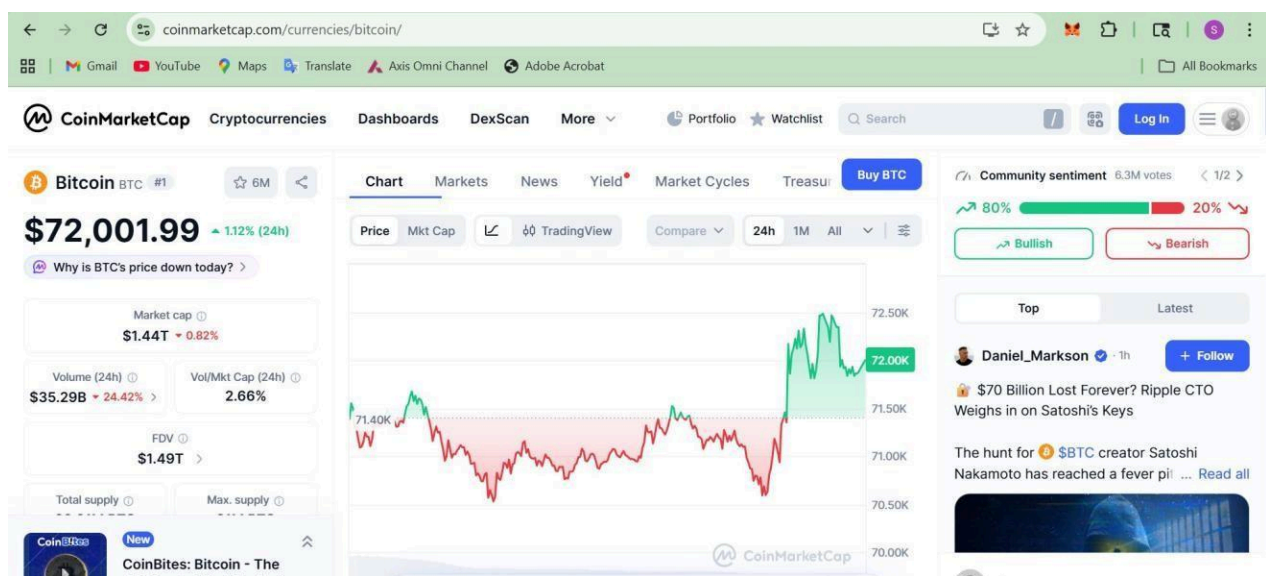
coingecko

CoinMarketCap website



This screenshot shows the top cryptocurrencies listed on the CoinMarketCap website. The table includes columns for rank, name, price, and percentage changes over 1h, 24h, and 7d. It also displays market capitalization, 24h volume, and circulating supply for each asset.

#	Name	Price	1h %	24h %	7d %	Market Cap	Volume(24h)	Circulating Supply
	CoinMarketCap 20 Index DTF	\$147.16	▲0.07%	▲1.08%	▲7.32%	\$15,198,584	\$6,929,640	103.27K CMC20
1	Bitcoin BTC	\$72,011.07	▲0.14%	▲1.06%	▲7.49%	\$1,441,204,730,589	\$35,222,223,723	20.01M BTC
2	Ethereum ETH	\$2,213.18	▲0.24%	▲0.34%	▲7.55%	\$267,111,520,131	\$16,449,534,711	120.69M ETH
3	Tether USDT	\$0.9999	▲0.02%	▲0.00%	▲0.01%	\$184,108,354,626	\$72,019,248,206	184.11B USDT
4	XRP XRP	\$1.35	▲0.32%	▲0.75%	▲4.09%	\$83,451,375,890	\$2,488,726,984	61.4B XRP



Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

EXPERIMENT 11

Aim - To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory -

Hyperledger Fabric Network Setup

- Consortium & Membership Service Provider (MSP): Fabric is permissioned, meaning only authorized organizations can join. MSPs define identities and roles.
- Peers: Nodes that maintain the ledger and endorse transactions. Each organization typically runs peers.
- Orderer: The ordering service packages endorsed transactions into blocks and delivers them to peers.
- Ledger: Composed of a blockchain (immutable transaction log) and a state database (current values).
- Channels: Private sub-networks where a subset of organizations transact confidentially.

Chaincode (Smart Contracts)

- Chaincode defines the business logic for asset management.
- Written in Go, Java, or Node.js.
- Deployed to peers, then instantiated on a channel.
- Functions typically include createAsset, queryAsset, transferAsset, and deleteAsset.

Transaction Flow

1. Proposal Creation: A client application submits a transaction proposal (e.g., create asset).
2. Endorsement: Peers simulate the transaction using chaincode and return signed results.
3. Ordering: The orderer sequences endorsed transactions into blocks.
4. Validation & Commit: Peers validate endorsements, check for conflicts, and commit the block to the ledger.

Asset Operations

- Asset Creation:
 - Chaincode function createAsset stores asset attributes (ID, owner, value) in the ledger's state database.
- Asset Querying:
 - queryAsset retrieves asset details by key. Rich queries are possible if CouchDB is used as the state database.
- Asset Transfer:
 - transferAsset updates the owner field of an asset. The transaction is endorsed, ordered, and committed, ensuring immutability and consensus.

Key Principles

- Immutability: Once committed, transactions cannot be altered.

- Consensus: Endorsement policies ensure multiple organizations agree before a transaction is valid.
- Privacy: Channels and private data collections allow selective visibility.
- Trust: Identities are cryptographically verified via MSPs.

In essence, Hyperledger Fabric provides a modular, permissioned blockchain framework where chaincode governs asset lifecycles. The theory emphasizes identity-driven trust, modular architecture, and consensus-based validation — ensuring secure creation, querying, and transfer of assets across organizations.

Prerequisite: Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:

wsl --list --verbose

If it shows **no distributions**, you must install Ubuntu.

Install Ubuntu for WSL

Run this command in **PowerShell (Admin)**:

wsl --install -d Ubuntu

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.

Restart Your Computer

After installation finishes, **restart your PC**.

Open Ubuntu

Open:

Start Menu → Ubuntu

It will ask you to
create:

Username

Password

Now Ubuntu is ready.

Implementation

Step 1 — Update the system

code :

sudo apt update

sudo apt upgrade -y

Explain: `apt update` refreshes the package index — it tells your system what versions of software are available. `apt upgrade` installs the newer versions. Fabric's dependencies (Docker, curl, git) can fail or behave unexpectedly on outdated system libraries. Always do this first on a fresh machine.

```
pranav@DESKTOP-OSHSLJ1:~$ sudo apt update
sudo apt upgrade -y
[sudo] password for pranav:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1540 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [246 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [18.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [976 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [219 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.8 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2683 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [623 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6732 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:28 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1837 kB]
Get:30 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [336 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
```

Step 2 — Verify prerequisites

code :

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --version
```

```
docker ps
```

```
pranav@DESKTOP-OSHSLJ1:~$ node -v
v18.19.1
pranav@DESKTOP-OSHSLJ1:~$ npm -v
11.8.0
pranav@DESKTOP-OSHSLJ1:~$ git --version
git version 2.43.0
pranav@DESKTOP-OSHSLJ1:~$ docker --version
docker ps
Docker version 29.2.0, build 0b9d198
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Explain: Fabric requires Docker (to run peer/orderer containers), Git (to clone the repo), and Node.js + npm (to build JavaScript chaincode). Running `docker ps` with no output confirms the Docker daemon is running and your user can access it. If `docker ps` requires sudo, you need to add your user to the docker group.

Step 3 — Clone fabric-samples

code :

```
git clone https://github.com/hyperledger/fabric-samples
```

```
cd fabric-samples
```

```
pranav@DESKTOP-OSHSLJ1:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15527, done.
remote: Counting objects: 100% (98/98), done.
remote: Compressing objects: 100% (60/60), done.
remote: Total 15527 (delta 57), reused 38 (delta 38), pack-reused 15429 (from 3)
Receiving objects: 100% (15527/15527), 24.16 MiB | 6.18 MiB/s, done.
Resolving deltas: 100% (8722/8722), done.
pranav@DESKTOP-OSHSLJ1:~$ cd fabric-samples
```

Explain: `fabric-samples` is the official repository containing the test-network scripts, ready-made chaincode examples (including `asset-transfer-basic`), and configuration files. Everything else in this guide runs from inside this directory.

Step 4 — Download the install script and run it
code :

```
curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker
```

```
pranav@DESKTOP-OSHSLJ1:/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
pranav@DESKTOP-OSHSLJ1:/fabric-samples$ chmod +x install-fabric.sh
pranav@DESKTOP-OSHSLJ1:/fabric-samples$ ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /home/pranav/fabric-samples
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Done    0     0     0    0      0      0      0      0
100 120M  100 120M    0     0  6272k    0  0:00:19  0:00:19 --:--:-- 6483k
==> Done.
```

```
====> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15        86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15      9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15         3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                  654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15                654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest                654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                       453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                    453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                    453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                    86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15                 86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest                 86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                  9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15               9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest               9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                     3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                  3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                  3e25adc31a75      230MB      66.2MB
```



```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ ./install-fabric.sh docker

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faeala79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
```

Explain: This one script does three things simultaneously. `docker` pulls all Fabric Docker images — `fabric-peer`, `fabric-orderer`, `fabric-ccenv`, `fabric-baseos`, `fabric-ca`, and critically `fabric-nodeenv` (needed to build Node.js chaincode). `binary` downloads the CLI tools — `peer`, `configtxgen`, `configtxlator` — into `~/fabric-samples/bin`. `samples` ensures the chaincode sample directories are present. Without `fabric-nodeenv`, chaincode deployment fails later.

Step 5 — Verify Docker images are present

code :

`docker images | grep fabric`

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB   U
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-baseos:2.5                  654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                   453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-nodeenv:2.5                17e2d447ca0d      570MB      146MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB   U
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB      49.5MB   U
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB   U
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB   U
```

Explain: Confirm `hyperledger/fabric-nodeenv:2.5` is in the list. This image is the Node.js build environment that Fabric uses to compile and run JavaScript chaincode inside a container. If it is missing, `deployCC` will fail with a "No such image" error.

Step 6 — Set PATH and config path

code :

`export PATH=$PATH:$HOME/fabric-samples/bin`

`export FABRIC_CFG_PATH=$HOME/fabric-samples/config`

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger

```

Explain: The `peer` binary lives in `~/fabric-samples/bin`, which is not on your shell's PATH by default. Without this export, the shell cannot find the command and returns "peer not found". `FABRIC_CFG_PATH` tells the peer CLI where to find `core.yaml`, the main configuration file. Without it, the peer errors with "cannot init crypto, path does not exist".

code :

`peer version`

You should see `Version: v2.5.15`.

Step 7 — Start the network and create the channel

code :

`cd ~/fabric-samples/test-network`

`./network.sh up createChannel`

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ cd ~/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org1.example.com      Removed
✓Container orderer.example.com         Removed
✓Container peer0.org2.example.com      Removed
✓Network fabric_test                  Removed
✓Volume compose_orderer.example.com   Removed
✓Volume compose_peer0.org1.example.com Removed
✓Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59f864cfaf1398b52d341cb:latest
Deleted: sha256:27056269ed9759828700d7bfc21149ac76c17d725ae6f9a12c07cc42a04f539f163835f604aa885d60dc:latest
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59f864cfaf1398b52d341cb:latest
Deleted: sha256:f7c74fe5fd5a6a73117e649e923a5de0a595771f793ee8e1b0047fc35d2d315c

```



```

pranav@DESKTOP-USHSLJ1:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using dat
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/pranav/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[*] up 7/7
✓Network fabric_test Created
✓Volume compose_orderer.example.com Created
✓Volume compose_peer0.org1.example.com Created
✓Volume compose_peer0.org2.example.com Created
✓Container peer0.org1.example.com Created
✓Container orderer.example.com Created
✓Container peer0.org2.example.com Created
CONTAINER ID IMAGE COMMAND CREATED STATUS

```

Explain: This single command does everything needed to have a running network with a channel. Internally it uses **cryptogen** to generate TLS certificates and identity material (MSP) for Org1, Org2, and the Orderer. It starts three Docker containers — **orderer.example.com** on port 7050, **peer0.org1.example.com** on port 7051, and **peer0.org2.example.com** on port 9051. It then runs **configtxgen** to create **mychannel.block** (the channel genesis block), submits it to the orderer, joins both peers to the channel, and sets anchor peers so the two organizations can discover each other via gossip. The default channel name is **mychannel**.

Step 8 — Deploy the chaincode

code :

```

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```

```

pranav@DESKTOP-USHSLJ1:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang
+ res=0
Chaincode is packaged
Installation chaincode on peer0 peer1

```

```

2026-03-17 16:28:58.033 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:7051
2026-03-17 16:28:58.037 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organisation 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organisation 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

```

Explain: `-ccn basic` is the name your chaincode will be called by in all future commands. `-ccp` is the path to the chaincode source code. `-ccl javascript` tells Fabric to use the Node.js runtime. Internally this runs the full Fabric chaincode lifecycle — package the source into a `.tar.gz`, install it on both peers, have Org1 approve the definition, have Org2 approve the definition (both approvals are required since majority endorsement is the default policy), and finally commit the definition to the channel. Once committed, two new Docker containers appear — one chaincode container per peer (`dev-peer0.org1...` and `dev-peer0.org2...`).

Step 9 — Set peer identity environment variables code :

```

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

```

pranav@DESKTOP-0SHSLJI:~/fabric-samples/test-network$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

Explain: The `peer` CLI has no built-in session — it does not know who you are. Every command needs to know which organization's identity to use, which peer to connect to, and how to verify TLS. These five variables provide exactly that. `CORE_PEER_LOCALMSPID` declares "I am Org1". `CORE_PEER_MSPCONFIGPATH` points to the Admin user's private signing key and certificate. `CORE_PEER_TLS_ROOTCERT_FILE` provides the CA certificate to verify the

peer's TLS identity. **CORE_PEER_ADDRESS** is which peer to send requests to. These must be re-exported in every new terminal session.

Step 10 — Initialize the ledger with sample data

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile   
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles   
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles   
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'
```

```
pranav@DESKTOP-05HSLJ11:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscac  
erts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.co  
m/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.co  
m/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'  
2026-03-17 16:37:35.757 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:9051  
2026-03-17 16:37:35.757 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:7051  
2026-03-17 16:37:35.758 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Explain: **invoke** is used for any operation that writes to the ledger. **-o localhost:7050** is the orderer address. **--tls** and **--cafile** enable TLS and provide the CA cert to verify the orderer. **-C mychannel** and **-n basic** identify which channel and chaincode. The two **--peerAddresses** and **--tlsRootCertFiles** pairs are the critical part — the default endorsement policy requires a majority of organizations to endorse. With two orgs, both must sign. Sending to only one peer results in **ENDORSEMENT_POLICY_FAILURE**. **--waitForEvent** blocks until both peers confirm the

transaction is committed (status VALID) rather than just submitted. **InitLedger** seeds the ledger with six sample assets.

Step 11 — Query all assets

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'  
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explain: **query** is for read-only operations. No transaction is created, no block is written, no endorsement from multiple peers is needed. It is fast and cheap. This returns all six assets seeded by **InitLedger**:

```
[ {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},  
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},  
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},  
  {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},  
  {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},  
  {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800} ]
```

Step 12 — Read a specific asset

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset5"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'  
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Result:

```
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15}
```

Explain: **ReadAsset** takes an asset ID and returns that single asset's data from the world state. The world state is a key-value database that stores the current (latest) value for each asset. The full transaction history is stored in the blockchain, but queries read from this faster world state.

Step 13 — Create a new asset

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tls  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'  
2026-03-17 16:38:38.378 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:7051  
2026-03-17 16:38:38.371 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:9051  
2026-03-17 16:38:38.371 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200  
{"Color":"blue","Size":10,"Owner":"Pranav","AppraisedValue":1000}
```

Explain: **CreateAsset** takes five arguments in order — [ID, Color, Size, Owner, AppraisedValue]. This is a write operation so both peers must endorse. The orderer packages the endorsed transaction into a block and delivers it to both peers. Both confirm VALID and the asset is permanently recorded on the ledger.

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'  
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}  
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}
```


The transaction flow showing what happens inside Fabric every time you run **invoke** or **query** Shows what happens inside Fabric every time you run **peer chaincode invoke** — the proposal, endorsement, ordering, and commit flow — and why both peers are always required for any write operation.

Step 14 — Transfer asset ownership code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'
```

```
pranav@DESKTOP-DSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.c  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'  
[2026-03-17 16:41:28.692 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [e834ab102eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
status (VALID) at localhost:7051  
[2026-03-17 16:41:28.692 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [e834ab102eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
status (VALID) at localhost:7051  
[2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
```

Explain: **TransferAsset** takes [**assetID**, **newOwner**]. It reads the current asset from the world state, updates the Owner field, and writes it back. The function returns the previous owner ("**Pranav**") as confirmation. Both peers endorse, the orderer commits the block, and the ledger is updated permanently.

Verify the ownership change:

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

```
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

Owner is now **Rahul**

Step 15 — To stop/bring down the Hyperledger Fabric network, run:

[./network.sh](#) down

```
pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container orderer.example.com          Removed
✓Container peer0.org2.example.com       Removed
✓Container peer0.org1.example.com       Removed
✓Network fabric_test                   Removed
✓Volume compose_peer0.org1.example.com Removed
✓Volume compose_peer0.org2.example.com Removed
✓Volume compose_orderer.example.com    Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035f864cfa1398b52d341cb:latest
Deleted: sha256:aa29d3b9fb04e6316131fbf334a23f28d8a1646fa9d40090916c02d54bc5b76d
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035163835f604aa885d60dc:latest
Deleted: sha256:08d49c295856f950367880744bcacfa56a7134bf208cacfab8a29ef9221703c
```

Conclusion

The Hyperledger Fabric network was successfully implemented, demonstrating asset creation, querying, and transfer using chaincode. The process highlights key concepts such as consensus, immutability, and secure transaction flow. Overall, it provides a clear understanding of permissioned blockchain systems and their real-world applications in asset management.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	03
Name	Bhagyesh Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Blockchain-Based Voting System

Secure, transparent, and tamper-proof electronic voting using distributed ledger technology

Group 2 :

Prathamesh Palve D20A_2

Bhagyesh Patil D20A_3

Mohit Patil D20A_4

Vedant Sanap D20A_50

Introduction to Digital Voting

What is Voting?

Voting is the fundamental process of democratic decision-making where eligible participants express their preferences to elect representatives or decide policies.

Problems in Traditional Systems

- Election fraud and ballot stuffing
- Lack of transparency in counting
- Manual errors in vote tallying
- Difficult audit and verification
- High operational costs

What is Blockchain?

A distributed digital ledger that records transactions across multiple computers in a way that makes them secure, transparent, and immutable.



Decentralization

Data stored across multiple nodes instead of single authority



Transparency

All transactions visible to authorized participants



Immutability

Once recorded, data cannot be altered or deleted



Problem Statement

Trust Deficit

Current systems lack mechanisms to verify vote integrity without manual recounts

Security Vulnerabilities

Centralized databases can be hacked, manipulated, or corrupted by malicious actors

Need for Innovation

Modern democracy requires secure, transparent, and accessible voting infrastructure

Proposed Solution

A blockchain-based voting system leverages distributed ledger technology to create an **immutable, transparent, and secure** voting platform that eliminates fraud while maintaining voter privacy.



System Architecture



Admin

Election administrator manages candidates and election parameters



Voter

Registered voters authenticate and cast encrypted votes



Smart Contract

Ethereum-based contract executes voting logic and stores votes



Results

Real-time, verifiable election results displayed to all participants

Smart Contracts Implementation

Voting Contract

Core smart contract written in Solidity that manages the entire voting process:

- Stores candidate list and vote counts
- Records voter addresses to prevent duplicate voting
- Tallies votes automatically
- Provides read-only access to results

Admin Control Contract

Separate contract for election management:

- Adds or removes candidates from ballot
- Starts and ends voting period
- Resolves disputes if necessary
- Maintains election integrity

Working Process



Admin Adds Candidates

Election administrator deploys contract and populates ballot with eligible candidates



Voters Register

Eligible voters register using unique credentials verified by the system



Votes Cast

Authenticated voters submit encrypted votes recorded on blockchain



Blockchain Storage

Each vote becomes immutable transaction visible to authorized auditors



Results Displayed

Real-time results update automatically as votes are counted

Technologies & Implementation



Solidity

Object-oriented programming language for writing Ethereum smart contracts



Remix IDE

Web-based integrated development environment for compiling and testing contracts



MetaMask / Ganache

Wallet interface connects frontend to blockchain; Ganache provides local test network



Ethereum Blockchain

Distributed ledger platform executing smart contracts and storing vote records

Advantages, Limitations & Future

Advantages

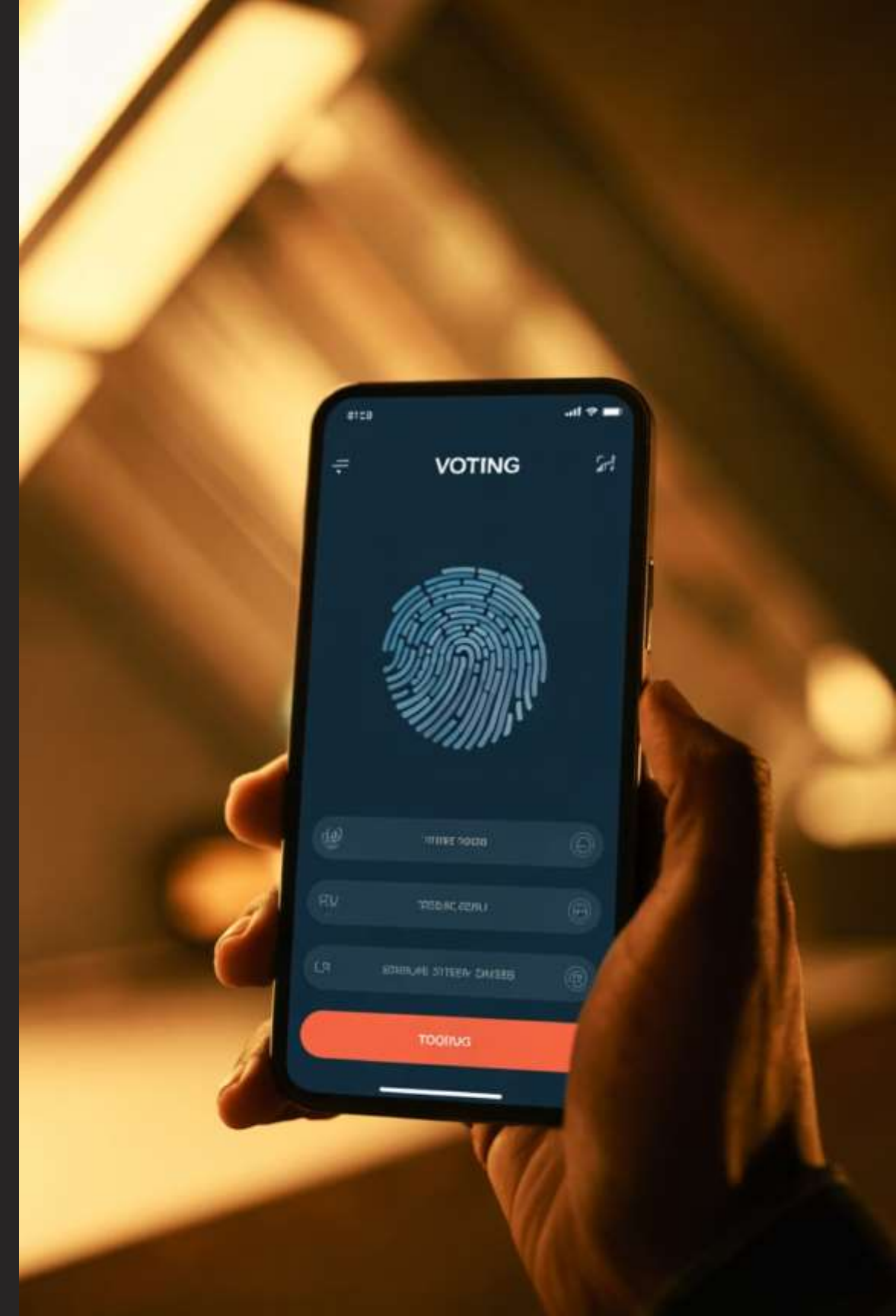
- **Transparency:** Anyone can verify vote counts
- **Security:** Cryptographic protection prevents tampering
- **Immutability:** Once recorded, votes cannot be altered
- **Decentralization:** No single point of failure

Limitations

- Requires internet connectivity
- Gas fees for blockchain transactions
- Technical complexity for non-technical users
- Scalability concerns with large elections

Future Enhancements

Mobile voting applications, biometric authentication, integration with government electoral systems, zero-knowledge proofs for enhanced privacy



Thankyou